```
// features/squat/visual_scoring.py
from dataclasses import dataclass


@dataclass(frozen=True)
class VisualScoringThresholds:
    knee_sway_ratio: float = 0.08
    knee_valgus_angle: float = 10.0
    center_deviation_ratio: float = 0.10
    left_right_symmetry: float = 0.85
    linkage_smoothness: float = 0.70
    squat_depth_ratio: float = 0.70


@dataclass
class VisualScoringBreakdown:
    front_score: int
    side_score: int
    findings: list[str]
    suggestions: list[str]


DEFAULT_VISUAL_THRESHOLDS = VisualScoringThresholds()


def clamp01(value: float) -> float:
    return max(0.0, min(1.0, value))


def calculate_front_view_score(
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    left_right_symmetry: float,
) -> int:
    sway_score = clamp01(1 - knee_sway_ratio / 0.12)
    valgus_score = clamp01(1 - knee_valgus_angle / 20.0)
    center_score = clamp01(1 - center_deviation_ratio / 0.15)
    symmetry_score = clamp01(left_right_symmetry)

    score = 100 * (
        sway_score * 0.30
        + valgus_score * 0.25
        + center_score * 0.20
        + symmetry_score * 0.25
    )

    return round(score)


def calculate_side_view_score(
    linkage_smoothness: float,
    squat_depth_ratio: float,
) -> int:
    linkage_score = clamp01(linkage_smoothness)
    depth_score = clamp01(squat_depth_ratio)

    score = 100 * (
        linkage_score * 0.55
        + depth_score * 0.45
    )

    return round(score)


def build_findings(
    *,
```

```
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    left_right_symmetry: float,
    linkage_smoothness: float,
    squat_depth_ratio: float,
    thresholds: VisualScoringThresholds = DEFAULT_VISUAL_THRESHOLDS,
) -> list[str]:
    findings: list[str] = []

    if knee_sway_ratio > thresholds.knee_sway_ratio:
        findings.append("膝部左右晃动较明显")

    if knee_valgus_angle > thresholds.knee_valgus_angle:
        findings.append("膝部内扣趋势明显")

    if center_deviation_ratio > thresholds.center_deviation_ratio:
        findings.append("重心存在单侧偏移")

    if left_right_symmetry < thresholds.left_right_symmetry:
        findings.append("左右发力与节奏不够对称")

    if linkage_smoothness < thresholds.linkage_smoothness:
        findings.append("髋膝踝联动不够顺畅")

    if squat_depth_ratio < thresholds.squat_depth_ratio:
        findings.append("下蹲深度不足")

    return findings


def build_suggestions(
    *,
    knee_sway_ratio: float,
    knee_valgus_angle: float,
    center_deviation_ratio: float,
    left_right_symmetry: float,
    linkage_smoothness: float,
    squat_depth_ratio: float,
    thresholds: VisualScoringThresholds = DEFAULT_VISUAL_THRESHOLDS,
) -> list[str]:
    suggestions: list[str] = []

    if knee_sway_ratio > thresholds.knee_sway_ratio:
        suggestions.append("膝部左右晃动较明显，优先加强下肢稳定与节奏控制。")

    if knee_valgus_angle > thresholds.knee_valgus_angle:
        suggestions.append("存在膝内扣趋势，注意膝盖方向与脚尖方向保持一致。")

    if center_deviation_ratio > thresholds.center_deviation_ratio:
        suggestions.append("重心存在单侧偏移，注意下蹲和起身过程保持身体居中。")

    if left_right_symmetry < thresholds.left_right_symmetry:
        suggestions.append("左右发力不够对称，建议增加单侧控制训练。")

    if linkage_smoothness < thresholds.linkage_smoothness:
        suggestions.append("髋膝踝联动不够顺畅，建议先放慢节奏巩固动作模式。")

    if squat_depth_ratio < thresholds.squat_depth_ratio:
        suggestions.append("下蹲深度不足，建议优先改善髋膝协同和踝活动度。")

    if not suggestions:
        suggestions.append("本次视觉评分显示动作整体稳定，可在下一阶段加入节奏控制或轻负重进阶。")

    return suggestions[:3]
```

```
// features/squat/live_analysis.py
from dataclasses import dataclass
from threading import Lock
from time import monotonic_ns
from uuid import uuid4

from app.core.errors import NotFoundError
from app.features.squat.pose_contract import (
    MIN_KEYPOINT_VISIBILITY,
    REQUIRED_ANALYSIS_POINTS,
)
from app.features.squat.schemas import CameraViewMode, LiveSquatMetricsResponse

Point = tuple[float, float]
TRACKED_POINTS = REQUIRED_ANALYSIS_POINTS
VIEW_MODES: tuple[CameraViewMode, CameraViewMode] = ("front", "side")


@dataclass
class LiveAnalysisState:
    phase: str = "up"
    rep_count: int = 0
    smoothed_metrics: dict[str, float] | None = None
    previous_points: dict[str, Point] | None = None
    last_rep_timestamp_ms: int | None = None
    tempo_seconds: float | None = None


class LiveSquatSessionStore:
    def __init__(self) -> None:
        self._sessions: dict[str, dict[CameraViewMode, LiveAnalysisState]] = {}
        self._lock = Lock()

    def create_session(self) -> str:
        session_id = f"live-{uuid4().hex[:12]}"
        with self._lock:
            self._sessions[session_id] = {
                view_mode: LiveAnalysisState()
                for view_mode in VIEW_MODES
            }
        return session_id

    def close_session(self, session_id: str) -> None:
        with self._lock:
            self._sessions.pop(session_id, None)

    def clear(self) -> None:
        with self._lock:
            self._sessions.clear()

    def update(
        self,
        session_id: str,
        view_mode: CameraViewMode,
        keypoints: list[object],
        *,
        now_ms: int | None = None,
    ) -> LiveSquatMetricsResponse | None:
        with self._lock:
            session_views = self._sessions.get(session_id)
            if session_views is None:
                raise NotFoundError(f"Live camera session not found: {session_id}")

            state = session_views[view_mode]
            return derive_live_metrics(
                keypoints=keypoints,
                state=state,
```

```
                now_ms=now_ms or monotonic_ns() // 1_000_000,
            )


def collect_missing_required_points(keypoints: list[object]) -> list[str]:
    visible_points = build_visible_point_map(keypoints)
    return [
        name for name in REQUIRED_ANALYSIS_POINTS
        if name not in visible_points
    ]


def derive_live_metrics(
    *,
    keypoints: list[object],
    state: LiveAnalysisState,
    now_ms: int,
) -> LiveSquatMetricsResponse | None:
    visible_points = build_visible_point_map(keypoints)
    if any(name not in visible_points for name in REQUIRED_ANALYSIS_POINTS):
        return None

    left_hip = visible_points["left_hip"]
    right_hip = visible_points["right_hip"]
    left_knee = visible_points["left_knee"]
    right_knee = visible_points["right_knee"]
    left_ankle = visible_points["left_ankle"]
    right_ankle = visible_points["right_ankle"]
    left_shoulder = visible_points.get("left_shoulder")
    right_shoulder = visible_points.get("right_shoulder")

    hip_width = distance(left_hip, right_hip)
    shoulder_width = distance(left_shoulder, right_shoulder) if left_shoulder and right_shoulder else 0.0
    width_scale = max(hip_width, shoulder_width, 0.12)

    left_knee_angle = angle_at(left_hip, left_knee, left_ankle)
    right_knee_angle = angle_at(right_hip, right_knee, right_ankle)
    mean_knee_angle = average([left_knee_angle, right_knee_angle])

    raw_depth = clamp01((175.0 - mean_knee_angle) / 95.0)
    raw_knee_sway_ratio = clamp01(
        average([
            abs(left_knee[0] - left_ankle[0]),
            abs(right_knee[0] - right_ankle[0]),
        ]) / (width_scale * 4.0)
    )
    raw_knee_valgus_angle = round_to_two_decimals(
        clamp01(
            max(
                abs(left_knee[0] - left_ankle[0]),
                abs(right_knee[0] - right_ankle[0]),
            ) / (width_scale * 0.6)
        ) * 30.0
    )

    shoulder_center = midpoint(left_shoulder, right_shoulder) if left_shoulder and right_shoulder else midpo
    hip_center = midpoint(left_hip, right_hip)
    ankle_center = midpoint(left_ankle, right_ankle)
    body_center = midpoint(shoulder_center, hip_center)
    raw_center_deviation_ratio = clamp01(abs(body_center[0] - ankle_center[0]) / (width_scale * 1.2))
    raw_left_right_symmetry = clamp01(1.0 - abs(left_knee_angle - right_knee_angle) / 60.0)

    tracked_points = {name: visible_points[name] for name in TRACKED_POINTS}
    raw_linkage_smoothness = calculate_linkage_smoothness(
        previous_points=state.previous_points,
        current_points=tracked_points,
        width_scale=width_scale,
```

```
    )

    next_metrics = {
        "knee_sway_ratio": raw_knee_sway_ratio,
        "knee_valgus_angle": raw_knee_valgus_angle,
        "center_deviation_ratio": raw_center_deviation_ratio,
        "left_right_symmetry": raw_left_right_symmetry,
        "linkage_smoothness": raw_linkage_smoothness,
        "squat_depth_ratio": raw_depth,
    }

    state.smoothed_metrics = (
        smooth_metric_set(state.smoothed_metrics, next_metrics)
        if state.smoothed_metrics
        else next_metrics
    )
    state.previous_points = tracked_points
    update_rep_state(state=state, depth_ratio=raw_depth, now_ms=now_ms)

    return LiveSquatMetricsResponse(
        squat_count=state.rep_count,
        knee_sway_ratio=state.smoothed_metrics["knee_sway_ratio"],
        knee_valgus_angle=state.smoothed_metrics["knee_valgus_angle"],
        center_deviation_ratio=state.smoothed_metrics["center_deviation_ratio"],
        left_right_symmetry=state.smoothed_metrics["left_right_symmetry"],
        linkage_smoothness=state.smoothed_metrics["linkage_smoothness"],
        squat_depth_ratio=state.smoothed_metrics["squat_depth_ratio"],
        tempo_seconds=state.tempo_seconds,
    )


def build_visible_point_map(keypoints: list[object]) -> dict[str, Point]:
    visible_points: dict[str, Point] = {}
    for keypoint in keypoints:
        visibility = float(getattr(keypoint, "visibility", 1.0))
        if visibility < MIN_KEYPOINT_VISIBILITY:
            continue

        visible_points[str(getattr(keypoint, "name"))] = (
            float(getattr(keypoint, "x")),
            float(getattr(keypoint, "y")),
        )

    return visible_points


def calculate_linkage_smoothness(
    *,
    previous_points: dict[str, Point] | None,
    current_points: dict[str, Point],
    width_scale: float,
) -> float:
    if not previous_points:
        return 0.82

    deltas = [
        distance(previous_points[name], current_points[name]) / width_scale
        for name in TRACKED_POINTS
    ]
    mean_delta = average(deltas)
    spread = average([abs(delta - mean_delta) for delta in deltas])
    return clamp01(1.0 - spread / 0.12)


def update_rep_state(*, state: LiveAnalysisState, depth_ratio: float, now_ms: int) -> None:
    if state.phase == "up" and depth_ratio >= 0.58:
        state.phase = "down"
```

```
        return

    if state.phase == "down" and depth_ratio <= 0.3:
        state.phase = "up"
        state.rep_count += 1

        if state.last_rep_timestamp_ms is not None:
            state.tempo_seconds = round_to_one_decimal((now_ms - state.last_rep_timestamp_ms) / 1000.0)

        state.last_rep_timestamp_ms = now_ms


def smooth_metric_set(previous: dict[str, float], current: dict[str, float]) -> dict[str, float]:
    return {
        key: smooth_value(previous[key], current[key])
        for key in current
    }


def smooth_value(previous: float, current: float, weight: float = 0.28) -> float:
    return previous * (1.0 - weight) + current * weight


def midpoint(first: Point, second: Point) -> Point:
    return (
        (first[0] + second[0]) / 2.0,
        (first[1] + second[1]) / 2.0,
    )


def angle_at(start: Point, pivot: Point, end: Point) -> float:
    start_vector = (start[0] - pivot[0], start[1] - pivot[1])
    end_vector = (end[0] - pivot[0], end[1] - pivot[1])
    dot = start_vector[0] * end_vector[0] + start_vector[1] * end_vector[1]
    magnitude = vector_magnitude(start_vector) * vector_magnitude(end_vector)
    if magnitude == 0:
        return 180.0

    cosine = max(-1.0, min(1.0, dot / magnitude))
    from math import acos, pi

    return acos(cosine) * (180.0 / pi)


def distance(first: Point | None, second: Point | None) -> float:
    if first is None or second is None:
        return 0.0
    return vector_magnitude((first[0] - second[0], first[1] - second[1]))


def vector_magnitude(vector: Point) -> float:
    from math import hypot

    return hypot(vector[0], vector[1])


def average(values: list[float]) -> float:
    return sum(values) / max(len(values), 1)


def clamp01(value: float) -> float:
    return max(0.0, min(1.0, value))


def round_to_one_decimal(value: float) -> float:
    return round(value, 1)
```

```
def round_to_two_decimals(value: float) -> float:
    return round(value, 2)
// features/squat/visual_detection.py
import logging
import shutil
import tempfile
import threading
import time
from dataclasses import dataclass, field
from pathlib import Path

import cv2
import mediapipe as mp
import numpy as np

from .pose_contract import MIN_KEYPOINT_VISIBILITY, POSE_CONNECTIONS

logger = logging.getLogger(__name__)

MODEL_PATH = Path(__file__).resolve().parent.parent.parent.parent.parent / "pose_landmarker.task"

try:
    from mediapipe.tasks import python
    from mediapipe.tasks.python import vision

    HAS_TASK_API = True
except ImportError:  # pragma: no cover - depends on local MediaPipe install
    python = None
    vision = None
    HAS_TASK_API = False


@dataclass
class KeypointData:
    name: str
    x: float
    y: float
    z: float
    visibility: float


@dataclass
class PoseFrame:
    keypoints: list[KeypointData] = field(default_factory=list)
    frame_width: int = 0
    frame_height: int = 0
    has_detection: bool = False


class PoseDetector:
    def __init__(
        self,
        model_path: Path | None,
        *,
        num_poses: int = 1,
        min_detection_confidence: float = 0.5,
        min_presence_confidence: float = 0.5,
        min_tracking_confidence: float = 0.5,
    ) -> None:
        self.backend = "mediapipe_tasks"
        self.model_path = model_path
        self._landmarker = None
        self._availability_detail = ""
        self._detection_lock = threading.Lock()
        self._last_timestamp_ms = 0
```

```
        self._num_poses = num_poses
        self._min_detection_confidence = min_detection_confidence
        self._min_presence_confidence = min_presence_confidence
        self._min_tracking_confidence = min_tracking_confidence

        self._initialize()

    @property
    def available(self) -> bool:
        return self._landmarker is not None

    @property
    def availability_detail(self) -> str:
        return self._availability_detail

    def process_frame(self, frame: np.ndarray, timestamp_ms: int | None = None) -> PoseFrame:
        if not self.available:
            raise RuntimeError(self.availability_detail)

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame_rgb)

        with self._detection_lock:
            resolved_timestamp = self._next_timestamp(timestamp_ms)
            result = self._landmarker.detect_for_video(mp_image, resolved_timestamp)

        return self._build_pose_frame(
            result=result,
            frame_width=frame.shape[1],
            frame_height=frame.shape[0],
        )

    def close(self) -> None:
        if self._landmarker is not None:
            self._landmarker.close()
            self._landmarker = None
            logger.info("PoseDetector closed")

    def draw_keypoints(self, frame: np.ndarray, pose_frame: PoseFrame) -> np.ndarray:
        if not pose_frame.has_detection:
            return frame

        visible_keypoints = {
            keypoint.name: keypoint
            for keypoint in pose_frame.keypoints
            if keypoint.visibility >= MIN_KEYPOINT_VISIBILITY
        }

        for start_name, end_name in POSE_CONNECTIONS:
            start = visible_keypoints.get(start_name)
            end = visible_keypoints.get(end_name)
            if start is None or end is None:
                continue

            cv2.line(
                frame,
                self._to_frame_point(start, frame.shape[1], frame.shape[0]),
                self._to_frame_point(end, frame.shape[1], frame.shape[0]),
                (20, 184, 166),
                2,
                lineType=cv2.LINE_AA,
            )

        for keypoint in visible_keypoints.values():
            point = self._to_frame_point(keypoint, frame.shape[1], frame.shape[0])
            cv2.circle(frame, point, 5, (255, 255, 255), -1, lineType=cv2.LINE_AA)
            cv2.circle(frame, point, 3, (245, 158, 11), -1, lineType=cv2.LINE_AA)
```

```
        return frame

    def _initialize(self) -> None:
        if not HAS_TASK_API:
            self._availability_detail = "MediaPipe Tasks API is unavailable."
            logger.warning(self._availability_detail)
            return

        if self.model_path is None:
            self._availability_detail = "Pose model path is not configured."
            logger.warning(self._availability_detail)
            return

        if not self.model_path.exists():
            self._availability_detail = f"Pose model not found: {self.model_path}"
            logger.warning(self._availability_detail)
            return

        try:
            cached_model_path = self._prepare_model_asset(self.model_path)
            options = vision.PoseLandmarkerOptions(
                base_options=python.BaseOptions(model_asset_path=str(cached_model_path)),
                running_mode=vision.RunningMode.VIDEO,
                num_poses=self._num_poses,
                min_pose_detection_confidence=self._min_detection_confidence,
                min_pose_presence_confidence=self._min_presence_confidence,
                min_tracking_confidence=self._min_tracking_confidence,
            )
            self._landmarker = vision.PoseLandmarker.create_from_options(options)
            self._availability_detail = "Pose detection ready."
            logger.info("PoseDetector initialized using %s", cached_model_path)
        except Exception as error:  # pragma: no cover - environment specific
            self._availability_detail = f"Pose detector unavailable: {error}"
            logger.warning(self._availability_detail)

    def _prepare_model_asset(self, model_path: Path) -> Path:
        cache_root = Path(tempfile.gettempdir()) / "squatlab-mediapipe"
        cache_root.mkdir(parents=True, exist_ok=True)
        cached_model_path = cache_root / model_path.name

        source_stat = model_path.stat()
        if not cached_model_path.exists():
            shutil.copyfile(model_path, cached_model_path)
            return cached_model_path

        cached_stat = cached_model_path.stat()
        if (
            cached_stat.st_size != source_stat.st_size
            or cached_stat.st_mtime_ns < source_stat.st_mtime_ns
        ):
            shutil.copyfile(model_path, cached_model_path)

        return cached_model_path

    def _next_timestamp(self, timestamp_ms: int | None) -> int:
        resolved = timestamp_ms or time.monotonic_ns() // 1_000_000
        if resolved <= self._last_timestamp_ms:
            resolved = self._last_timestamp_ms + 1
        self._last_timestamp_ms = resolved
        return resolved

    def _build_pose_frame(self, *, result, frame_width: int, frame_height: int) -> PoseFrame:
        pose_frame = PoseFrame(
            frame_width=frame_width,
            frame_height=frame_height,
            has_detection=False,
```

```
        )

        pose_landmarks = getattr(result, "pose_landmarks", None) or []
        if not pose_landmarks:
            return pose_frame

        primary_pose = pose_landmarks[0]
        pose_frame.has_detection = True

        for index, landmark in enumerate(primary_pose):
            pose_frame.keypoints.append(
                KeypointData(
                    name=vision.PoseLandmark(index).name.lower(),
                    x=float(landmark.x),
                    y=float(landmark.y),
                    z=float(landmark.z),
                    visibility=float(getattr(landmark, "visibility", 1.0)),
                )
            )

        return pose_frame

    @staticmethod
    def _to_frame_point(keypoint: KeypointData, frame_width: int, frame_height: int) -> tuple[int, int]:
        x = int(max(0.0, min(1.0, keypoint.x)) * frame_width)
        y = int(max(0.0, min(1.0, keypoint.y)) * frame_height)
        return (x, y)
```

```
// features/squat/service.py
from dataclasses import dataclass

from .schemas import SquatAssessmentRequest, SquatAssessmentResult
from .visual_scoring import (
    build_findings,
    build_suggestions,
    calculate_front_view_score,
    calculate_side_view_score,
)


@dataclass
class ScoringBreakdown:
    front_score: int
    side_score: int
    findings: list[str]
    suggestions: list[str]


class SquatAssessmentService:
    def score(self, payload: SquatAssessmentRequest, session_id: str) -> SquatAssessmentResult:
        breakdown = self._build_breakdown(payload)
        overall_score = round(
            breakdown.front_score * 0.50
            + breakdown.side_score * 0.50
        )
        summary = self._build_summary(
            findings=breakdown.findings,
            overall_score=overall_score,
            front_score=breakdown.front_score,
            side_score=breakdown.side_score,
        )

        return SquatAssessmentResult(
            session_id=session_id,
            overall_score=overall_score,
            front_score=breakdown.front_score,
            side_score=breakdown.side_score,
```

```
            findings=breakdown.findings,
            summary=summary,
            suggestions=breakdown.suggestions,
        )

    def _build_breakdown(self, payload: SquatAssessmentRequest) -> ScoringBreakdown:
        front_score = calculate_front_view_score(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
        )
        side_score = calculate_side_view_score(
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        )
        findings = build_findings(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        )
        suggestions = build_suggestions(
            knee_sway_ratio=payload.knee_sway_ratio,
            knee_valgus_angle=payload.knee_valgus_angle,
            center_deviation_ratio=payload.center_deviation_ratio,
            left_right_symmetry=payload.left_right_symmetry,
            linkage_smoothness=payload.linkage_smoothness,
            squat_depth_ratio=payload.squat_depth_ratio,
        )

        return ScoringBreakdown(
            front_score=front_score,
            side_score=side_score,
            findings=findings,
            suggestions=suggestions,
        )

    def _build_summary(
        self,
        *,
        findings: list[str],
        overall_score: int,
        front_score: int,
        side_score: int,
    ) -> str:
        if not findings:
            return (
                f"本次深蹲视觉评分为 {overall_score} 分，"
                f"正面得分 {front_score}，侧面得分 {side_score}，整体动作控制稳定。"
            )

        top_findings = "、".join(findings[:3])
        return (
            f"本次深蹲视觉评分为 {overall_score} 分，"
            f"正面得分 {front_score}，侧面得分 {side_score}，"
            f"主要观察到：{top_findings}。"
        )
```

```
// features/squat/repository.py
import json
from datetime import datetime, timezone

from app.core.errors import NotFoundError
from app.shared.db import DatabaseManager
```

```
from .schemas import ReportPreviewResponse, SessionSummary, SquatAssessmentRequest, SquatAssessmentResult


class SquatRepository:
    def __init__(self, db: DatabaseManager) -> None:
        self.db = db

    def save_assessment(self, payload: SquatAssessmentRequest, result: SquatAssessmentResult) -> None:
        created_at = datetime.now(timezone.utc).isoformat()

        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO squat_sessions (session_id, squat_count, overall_score, summary, created_at)
                VALUES (?, ?, ?, ?, ?)
                """,
                (
                    result.session_id,
                    payload.squat_count,
                    result.overall_score,
                    result.summary,
                    created_at,
                ),
            )
            connection.execute(
                """
                INSERT INTO squat_visual_assessments (
                    session_id,
                    front_score,
                    side_score,
                    findings,
                    suggestions
                )
                VALUES (?, ?, ?, ?, ?)
                """,
                (
                    result.session_id,
                    result.front_score,
                    result.side_score,
                    json.dumps(result.findings, ensure_ascii=False),
                    json.dumps(result.suggestions, ensure_ascii=False),
                ),
            )

    def list_sessions(self) -> list[SessionSummary]:
        with self.db.connect() as connection:
            rows = connection.execute(
                """
                SELECT session_id, squat_count, overall_score, summary, created_at
                FROM squat_sessions
                ORDER BY created_at DESC
                """
            ).fetchall()

        return [
            SessionSummary(
                session_id=row[0],
                squat_count=row[1],
                overall_score=row[2],
                summary=row[3],
                created_at=datetime.fromisoformat(row[4]),
            )
            for row in rows
        ]

    def build_report_preview(self, session_id: str) -> ReportPreviewResponse:
```

```
        with self.db.connect() as connection:
            visual_row = connection.execute(
                """
                SELECT s.session_id, s.summary, a.findings, a.suggestions
                FROM squat_sessions s
                JOIN squat_visual_assessments a ON a.session_id = s.session_id
                WHERE s.session_id = ?
                """,
                (session_id,),
            ).fetchone()

        if visual_row is None:
            raise NotFoundError(f"Session not found: {session_id}")

        return self._build_preview_response(
            session_id=visual_row[0],
            summary=visual_row[1],
            findings=json.loads(visual_row[2]),
            recommendations=json.loads(visual_row[3]),
        )

    def _build_preview_response(
        self,
        *,
        session_id: str,
        summary: str,
        findings: list[str],
        recommendations: list[str],
    ) -> ReportPreviewResponse:
        normalized_findings = findings or ["未观察到明显动作异常"]
        normalized_recommendations = recommendations or ["当前动作表现稳定，可继续保持现有训练节奏。"]

        return ReportPreviewResponse(
            session_id=session_id,
            title="深蹲视觉评分报告",
            summary=summary,
            findings=normalized_findings,
            recommendations=normalized_recommendations,
        )
```

```
// features/squat/schemas.py
from datetime import datetime
from typing import Literal
from uuid import uuid4

from pydantic import BaseModel, Field

CameraViewMode = Literal["front", "side"]
CameraReadinessState = Literal["no_detection", "insufficient_pose", "capturing", "ready"]


class SquatAssessmentRequest(BaseModel):
    squat_count: int = Field(ge=1, le=100)
    knee_sway_ratio: float = Field(ge=0, le=1)
    knee_valgus_angle: float = Field(ge=0, le=60)
    center_deviation_ratio: float = Field(ge=0, le=1)
    left_right_symmetry: float = Field(ge=0, le=1)
    linkage_smoothness: float = Field(ge=0, le=1)
    squat_depth_ratio: float = Field(ge=0, le=1)


class SquatAssessmentResult(BaseModel):
    session_id: str
    overall_score: int
    front_score: int
    side_score: int
    findings: list[str]
```

```
    summary: str
    suggestions: list[str]


class SessionSummary(BaseModel):
    session_id: str
    squat_count: int
    overall_score: int
    summary: str
    created_at: datetime


class ReportPreviewRequest(BaseModel):
    session_id: str


class ReportPreviewResponse(BaseModel):
    session_id: str
    title: str
    summary: str
    findings: list[str]
    recommendations: list[str]


class CameraStatusResponse(BaseModel):
    available: bool
    backend: str
    detail: str


class CameraAnalysisSessionResponse(BaseModel):
    session_id: str


class PoseKeypoint(BaseModel):
    name: str
    x: float
    y: float
    z: float
    visibility: float


class CameraKeypointsResponse(BaseModel):
    has_detection: bool
    frame_width: int
    frame_height: int
    keypoints: list[PoseKeypoint]
    detector_backend: str


class CameraFrameRequest(BaseModel):
    session_id: str = Field(min_length=1)
    view_mode: CameraViewMode
    frame_data_url: str = Field(min_length=1)


class LiveSquatMetricsResponse(BaseModel):
    squat_count: int = Field(ge=0)
    knee_sway_ratio: float = Field(ge=0, le=1)
    knee_valgus_angle: float = Field(ge=0, le=60)
    center_deviation_ratio: float = Field(ge=0, le=1)
    left_right_symmetry: float = Field(ge=0, le=1)
    linkage_smoothness: float = Field(ge=0, le=1)
    squat_depth_ratio: float = Field(ge=0, le=1)
    tempo_seconds: float | None = Field(default=None, ge=0)
```

```
class CameraFrameAnalysisResponse(CameraKeypointsResponse):
    session_id: str
    view_mode: CameraViewMode
    readiness_state: CameraReadinessState
    missing_keypoints: list[str]
    min_keypoint_visibility: float = Field(ge=0, le=1)
    pose_connections: list[tuple[str, str]]
    live_metrics: LiveSquatMetricsResponse | None = None


class HealthPayload(BaseModel):
    status: Literal["ok"] = "ok"
    service: str
    version: str = "1.0.0"


def build_session_id() -> str:
    return f"squat-{uuid4().hex[:12]}"
```

```
// features/squat/pose_contract.py
MIN_KEYPOINT_VISIBILITY = 0.35

POSE_CONNECTIONS = (
    ("left_shoulder", "right_shoulder"),
    ("left_shoulder", "left_elbow"),
    ("left_elbow", "left_wrist"),
    ("right_shoulder", "right_elbow"),
    ("right_elbow", "right_wrist"),
    ("left_shoulder", "left_hip"),
    ("right_shoulder", "right_hip"),
    ("left_hip", "right_hip"),
    ("left_hip", "left_knee"),
    ("right_hip", "right_knee"),
    ("left_knee", "left_ankle"),
    ("right_knee", "right_ankle"),
    ("left_ankle", "left_heel"),
    ("right_ankle", "right_heel"),
    ("left_heel", "left_foot_index"),
    ("right_heel", "right_foot_index"),
)

REQUIRED_ANALYSIS_POINTS = (
    "left_hip",
    "right_hip",
    "left_knee",
    "right_knee",
    "left_ankle",
    "right_ankle",
)
```

```
// features/screening/service.py
from datetime import datetime, timezone
from typing import Any

from .schemas import (
    CaptureQuality,
    CrossProtocolEvidence,
    IntegratedReportResponse,
    ProtocolResultResponse,
    ProtocolStatus,
    ProtocolType,
    build_report_id,
)

DISCLAIMER = "本报告用于姿态与动作风险筛查参考，不作为医学诊断依据。如筛查结果提示明显风险，建议由专业人员进一步评估。"


class ScreeningAnalysisService:
```

```
    def analyze_protocol(
        self,
        *,
        result_id: str,
        session_id: str,
        protocol: ProtocolType,
        capture_quality: CaptureQuality,
        metrics: dict[str, Any],
        now: datetime | None = None,
    ) -> ProtocolResultResponse:
        timestamp = now or datetime.now(timezone.utc)
        findings, risk_flags, recommendations = self._derive_protocol_output(
            protocol=protocol,
            capture_quality=capture_quality,
            metrics=metrics,
        )
        needs_recapture = capture_quality == "poor"
        needs_review = (not needs_recapture) and self._needs_protocol_review(protocol, metrics, risk_flags)
        status: ProtocolStatus = (
            "needs_recapture"
            if needs_recapture
            else "needs_review"
            if needs_review
            else "analyzed"
        )

        return ProtocolResultResponse(
            result_id=result_id,
            session_id=session_id,
            protocol=protocol,
            status=status,
            capture_quality=capture_quality,
            metrics=metrics,
            findings=findings,
            risk_flags=risk_flags,
            recommendations=recommendations,
            needs_recapture=needs_recapture,
            needs_review=needs_review,
            created_at=timestamp,
            updated_at=timestamp,
        )

    def build_integrated_report(
        self,
        *,
        session_id: str,
        results: list[ProtocolResultResponse],
        now: datetime | None = None,
    ) -> IntegratedReportResponse:
        timestamp = now or datetime.now(timezone.utc)
        if any(result.needs_recapture for result in results):
            return self._build_recapture_report(session_id=session_id, results=results, timestamp=timestamp)

        evidence = self._build_cross_protocol_evidence(results)
        adams_result = next((result for result in results if result.protocol == "adams_forward_bend"), None)
        adams_severity = str((adams_result.metrics if adams_result else {}).get("thoracic_asymmetry", "none"

        if adams_severity in {"moderate", "marked"} or any(result.needs_review for result in results):
            overall_risk = "review_required"
            next_action = "manual_review"
        elif evidence:
            overall_risk = "attention"
            next_action = "retest_later"
        elif any(result.risk_flags for result in results):
            overall_risk = "attention"
            next_action = "retest_later"
        else:
```

```
            overall_risk = "low"
            next_action = "pass"

        consistency_level = "multi_protocol_consistent" if evidence else ("single_protocol" if any(result.ri
        main_patterns = [item.pattern for item in evidence] or self._single_protocol_patterns(results)
        summary = self._build_summary(overall_risk=overall_risk, evidence=evidence, results=results)
        recommendations = self._build_report_recommendations(overall_risk=overall_risk, next_action=next_act

        return IntegratedReportResponse(
            report_id=build_report_id(),
            session_id=session_id,
            title="姿态与动作联合筛查报告",
            overall_risk=overall_risk,
            consistency_level=consistency_level,
            main_patterns=main_patterns,
            cross_protocol_evidence=evidence,
            next_action=next_action,
            summary=summary,
            recommendations=recommendations,
            disclaimer=DISCLAIMER,
            created_at=timestamp,
        )

    def _derive_protocol_output(
        self,
        *,
        protocol: ProtocolType,
        capture_quality: CaptureQuality,
        metrics: dict[str, Any],
    ) -> tuple[list[str], list[str], list[str]]:
        if capture_quality == "poor":
            return (
                ["采集质量不足，当前结果不适合生成正式筛查解释"],
                ["capture_quality_poor"],
                ["建议按引导重新采集后再生成报告。"],
            )

        if protocol == "static_posture":
            return self._derive_static_posture(metrics)
        if protocol == "adams_forward_bend":
            return self._derive_adams(metrics)
        return self._derive_squat(metrics)

    def _derive_static_posture(self, metrics: dict[str, Any]) -> tuple[list[str], list[str], list[str]]:
        findings: list[str] = []
        flags: list[str] = []
        direction = str(metrics.get("suspected_direction", "unclear"))

        if float(metrics.get("shoulder_height_diff_ratio", 0) or 0) >= 0.03:
            findings.append(("右" if direction == "right" else "左" if direction == "left" else "") + "肩高度不对称")
            flags.append(f"static_shoulder_asymmetry_{direction}")
        if float(metrics.get("pelvis_height_diff_ratio", 0) or 0) >= 0.03:
            findings.append("骨盆高度存在不对称")
            flags.append(f"static_pelvis_asymmetry_{direction}")
        if float(metrics.get("trunk_lateral_shift_ratio", 0) or 0) >= 0.04:
            findings.append(("右" if direction == "right" else "左" if direction == "left" else "") + "侧躯干偏移")
            flags.append(f"static_trunk_asymmetry_{direction}")

        return (
            findings or ["静态体态未见明显不对称"],
            flags,
            ["建议结合前屈筛查和动态动作结果进行综合判断。"],
        )

    def _derive_adams(self, metrics: dict[str, Any]) -> tuple[list[str], list[str], list[str]]:
        if not bool(metrics.get("forward_bend_completed", True)):
            return (
```

```
                ["Adams 前屈动作未完整完成"],
                ["adams_incomplete"],
                ["建议重新采集并保持峰值前屈 2-3 秒。"],
            )

        findings: list[str] = []
        flags: list[str] = []
        side = str(metrics.get("suspected_side", "unclear"))
        side_label = "右侧" if side == "right" else "左侧" if side == "left" else ""

        thoracic = str(metrics.get("thoracic_asymmetry", "none"))
        lumbar = str(metrics.get("lumbar_asymmetry", "none"))
        if thoracic in {"mild", "moderate", "marked"}:
            findings.append(f"胸段{side_label}不对称{self._severity_label(thoracic)}")
            flags.append(f"adams_thoracic_asymmetry_{side}")
        if lumbar in {"mild", "moderate", "marked"}:
            findings.append(f"腰段{side_label}不对称{self._severity_label(lumbar)}")
            flags.append(f"adams_lumbar_asymmetry_{side}")

        return (
            findings or ["Adams 前屈未见明显背部左右不对称"],
            flags,
            ["建议由专业人员复核本次前屈筛查证据。"] if flags else ["建议按周期复查。"],
        )

    def _derive_squat(self, metrics: dict[str, Any]) -> tuple[list[str], list[str], list[str]]:
        findings: list[str] = []
        flags: list[str] = []
        direction = str(metrics.get("dynamic_shift_direction", "unclear"))

        if float(metrics.get("center_deviation_ratio", 0) or 0) >= 0.06:
            findings.append(("右" if direction == "right" else "左" if direction == "left" else "") + "侧重心偏移")
            flags.append(f"dynamic_weight_shift_{direction}")
        if float(metrics.get("left_right_symmetry", 1) or 1) < 0.88:
            findings.append("左右发力与节奏不够对称")
            flags.append("dynamic_asymmetry")
        if float(metrics.get("knee_valgus_angle", 0) or 0) > 10:
            findings.append("膝部内扣趋势明显")
            flags.append("dynamic_knee_valgus")

        return (
            findings or ["深蹲动作控制整体稳定"],
            flags,
            ["建议结合静态体态结果观察是否存在同方向偏移。"] if flags else ["可按现有训练节奏继续观察。"],
        )

    def _needs_protocol_review(self, protocol: ProtocolType, metrics: dict[str, Any], risk_flags: list[str])
        if protocol != "adams_forward_bend":
            return False
        return str(metrics.get("thoracic_asymmetry", "none")) in {"moderate", "marked"} or len(risk_flags) >

    def _build_recapture_report(
        self,
        *,
        session_id: str,
        results: list[ProtocolResultResponse],
        timestamp: datetime,
    ) -> IntegratedReportResponse:
        poor_protocols = [result.protocol for result in results if result.needs_recapture]
        return IntegratedReportResponse(
            report_id=build_report_id(),
            session_id=session_id,
            title="姿态与动作联合筛查报告",
            overall_risk="recapture_needed",
            consistency_level="none",
            main_patterns=["capture_quality_poor"],
            cross_protocol_evidence=[],
```

```
            next_action="recapture",
            summary=f"{'、'.join(poor_protocols)} 采集质量不足，建议重新采集后再生成正式筛查解释。",
            recommendations=["请按采集引导重新完成质量不足的项目。"],
            disclaimer=DISCLAIMER,
            created_at=timestamp,
        )

    def _build_cross_protocol_evidence(self, results: list[ProtocolResultResponse]) -> list[CrossProtocolEvi
        right_evidence = self._collect_directional_evidence(results, "right")
        left_evidence = self._collect_directional_evidence(results, "left")
        evidence: list[CrossProtocolEvidence] = []
        if len(right_evidence) >= 2:
            evidence.append(CrossProtocolEvidence(
                pattern="trunk_asymmetry_right",
                protocols=[item[0] for item in right_evidence],
                direction="right",
                evidence=[item[1] for item in right_evidence],
                confidence="medium",
            ))
        if len(left_evidence) >= 2:
            evidence.append(CrossProtocolEvidence(
                pattern="trunk_asymmetry_left",
                protocols=[item[0] for item in left_evidence],
                direction="left",
                evidence=[item[1] for item in left_evidence],
                confidence="medium",
            ))
        return evidence

    def _collect_directional_evidence(
        self,
        results: list[ProtocolResultResponse],
        direction: str,
    ) -> list[tuple[ProtocolType, str]]:
        collected: list[tuple[ProtocolType, str]] = []
        for result in results:
            if any(flag.endswith(f"_{direction}") for flag in result.risk_flags):
                collected.append((result.protocol, result.findings[0]))
        return collected

    def _single_protocol_patterns(self, results: list[ProtocolResultResponse]) -> list[str]:
        patterns: list[str] = []
        for result in results:
            patterns.extend(result.risk_flags[:2])
        return patterns[:4]

    def _build_summary(
        self,
        *,
        overall_risk: str,
        evidence: list[CrossProtocolEvidence],
        results: list[ProtocolResultResponse],
    ) -> str:
        if overall_risk == "low":
            return "本次静态体态、Adams 前屈和深蹲动作结果未见明显一致性风险。"
        if evidence:
            first = evidence[0]
            return f"{'右侧' if first.direction == 'right' else '左侧'}相关不对称在多个筛查项目中均有体现，建议专业人员复核。"
        flagged = next((result for result in results if result.risk_flags), None)
        if flagged:
            return f"本次主要在 {flagged.protocol} 项目中观察到风险提示，建议后续复查观察。"
        return "本次筛查存在需要关注的项目，建议结合人工复核判断。"

    def _build_report_recommendations(self, *, overall_risk: str, next_action: str) -> list[str]:
        if next_action == "manual_review":
            return ["建议进行人工复核，确认本次筛查证据是否稳定。", "如复核仍提示明显风险，建议进一步专业评估。"]
        if next_action == "retest_later":
```

```
            return ["建议 4-8 周后复查，观察姿态和动作控制变化。"]
        if next_action == "pass":
            return ["建议按周期进行常规复查。"]
        return ["建议按系统提示处理本次筛查任务。"]

    def _severity_label(self, value: str) -> str:
        return {
            "mild": "轻度",
            "moderate": "较明显",
            "marked": "明显",
        }.get(value, "")
```

```
// features/screening/repository.py
import json
from datetime import datetime, timezone

from app.core.errors import NotFoundError
from app.shared.db import DatabaseManager

from .schemas import (
    IntegratedReportResponse,
    ProtocolProgress,
    ProtocolResultResponse,
    ProtocolType,
    ScreeningSessionCreateResponse,
    ScreeningSessionDetailResponse,
    ScreeningSessionSummary,
    SubjectCreateRequest,
    SubjectResponse,
    build_screening_session_id,
    build_subject_id,
)


class ScreeningRepository:
    def __init__(self, db: DatabaseManager) -> None:
        self.db = db

    def create_subject(self, payload: SubjectCreateRequest) -> SubjectResponse:
        created_at = datetime.now(timezone.utc)
        subject = SubjectResponse(
            subject_id=build_subject_id(),
            display_name=payload.display_name,
            sex=payload.sex,
            age=payload.age,
            height_cm=payload.height_cm,
            notes=payload.notes,
            created_at=created_at,
        )
        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO subjects (subject_id, display_name, sex, age, height_cm, notes, created_at)
                VALUES (?, ?, ?, ?, ?, ?, ?)
                """,
                (
                    subject.subject_id,
                    subject.display_name,
                    subject.sex,
                    subject.age,
                    subject.height_cm,
                    subject.notes,
                    subject.created_at.isoformat(),
                ),
            )
        return subject
```

```
    def create_session(self, *, subject_id: str, protocols: list[ProtocolType]) -> ScreeningSessionCreateRes
        self._ensure_subject_exists(subject_id)
        created_at = datetime.now(timezone.utc)
        session_id = build_screening_session_id()
        unique_protocols = list(dict.fromkeys(protocols))
        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO screening_sessions (session_id, subject_id, status, protocols, created_at, compl
                VALUES (?, ?, ?, ?, ?, NULL)
                """,
                (
                    session_id,
                    subject_id,
                    "in_progress",
                    json.dumps(unique_protocols, ensure_ascii=False),
                    created_at.isoformat(),
                ),
            )

        return ScreeningSessionCreateResponse(
            session_id=session_id,
            subject_id=subject_id,
            status="in_progress",
            protocols=[ProtocolProgress(protocol=protocol, status="not_started") for protocol in unique_prot
            created_at=created_at,
        )

    def get_session_detail(self, session_id: str) -> ScreeningSessionDetailResponse:
        session_row = self._get_session_row(session_id)
        results = self.list_protocol_results(session_id)
        return ScreeningSessionDetailResponse(
            session_id=session_row[0],
            subject_id=session_row[1],
            status=session_row[2],
            protocol_results=results,
            integrated_report=self.get_integrated_report_or_none(session_id),
            created_at=datetime.fromisoformat(session_row[4]),
            completed_at=datetime.fromisoformat(session_row[5]) if session_row[5] else None,
        )

    def list_sessions(self) -> list[ScreeningSessionSummary]:
        with self.db.connect() as connection:
            rows = connection.execute(
                """
                SELECT ss.session_id, ss.subject_id, s.display_name, ss.status,
                       ir.overall_risk, ir.next_action, ss.created_at, ss.completed_at
                FROM screening_sessions ss
                JOIN subjects s ON s.subject_id = ss.subject_id
                LEFT JOIN integrated_reports ir ON ir.session_id = ss.session_id
                ORDER BY ss.created_at DESC
                """
            ).fetchall()

        return [
            ScreeningSessionSummary(
                session_id=row[0],
                subject_id=row[1],
                subject_display_name=row[2],
                status=row[3],
                overall_risk=row[4],
                next_action=row[5],
                completed_protocols=[result.protocol for result in self.list_protocol_results(row[0])],
                created_at=datetime.fromisoformat(row[6]),
                completed_at=datetime.fromisoformat(row[7]) if row[7] else None,
            )
            for row in rows
```

```
        ]

    def save_protocol_result(self, result: ProtocolResultResponse) -> None:
        self._get_session_row(result.session_id)
        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO protocol_results (
                    result_id, session_id, protocol_type, status, capture_quality,
                    metrics, findings, risk_flags, recommendations,
                    needs_recapture, needs_review, created_at, updated_at
                )
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ON CONFLICT(session_id, protocol_type) DO UPDATE SET
                    status = excluded.status,
                    capture_quality = excluded.capture_quality,
                    metrics = excluded.metrics,
                    findings = excluded.findings,
                    risk_flags = excluded.risk_flags,
                    recommendations = excluded.recommendations,
                    needs_recapture = excluded.needs_recapture,
                    needs_review = excluded.needs_review,
                    updated_at = excluded.updated_at
                """,
                (
                    result.result_id,
                    result.session_id,
                    result.protocol,
                    result.status,
                    result.capture_quality,
                    json.dumps(result.metrics, ensure_ascii=False),
                    json.dumps(result.findings, ensure_ascii=False),
                    json.dumps(result.risk_flags, ensure_ascii=False),
                    json.dumps(result.recommendations, ensure_ascii=False),
                    int(result.needs_recapture),
                    int(result.needs_review),
                    result.created_at.isoformat(),
                    result.updated_at.isoformat(),
                ),
            )
        self._refresh_session_status(result.session_id)

    def list_protocol_results(self, session_id: str) -> list[ProtocolResultResponse]:
        with self.db.connect() as connection:
            rows = connection.execute(
                """
                SELECT result_id, session_id, protocol_type, status, capture_quality,
                       metrics, findings, risk_flags, recommendations,
                       needs_recapture, needs_review, created_at, updated_at
                FROM protocol_results
                WHERE session_id = ?
                ORDER BY created_at ASC
                """,
                (session_id,),
            ).fetchall()
        return [self._protocol_result_from_row(row) for row in rows]

    def save_integrated_report(self, report: IntegratedReportResponse) -> None:
        with self.db.connect() as connection:
            connection.execute(
                """
                INSERT INTO integrated_reports (
                    report_id, session_id, title, overall_risk, consistency_level,
                    main_patterns, cross_protocol_evidence, next_action, summary,
                    recommendations, disclaimer, created_at
                )
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

```
            ON CONFLICT(session_id) DO UPDATE SET
                report_id = excluded.report_id,
                title = excluded.title,
                overall_risk = excluded.overall_risk,
                consistency_level = excluded.consistency_level,
                main_patterns = excluded.main_patterns,
                cross_protocol_evidence = excluded.cross_protocol_evidence,
                next_action = excluded.next_action,
                summary = excluded.summary,
                recommendations = excluded.recommendations,
                disclaimer = excluded.disclaimer,
                created_at = excluded.created_at
            """,
            (
                report.report_id,
                report.session_id,
                report.title,
                report.overall_risk,
                report.consistency_level,
                json.dumps(report.main_patterns, ensure_ascii=False),
                json.dumps([item.model_dump() for item in report.cross_protocol_evidence], ensure_ascii=
                report.next_action,
                report.summary,
                json.dumps(report.recommendations, ensure_ascii=False),
                report.disclaimer,
                report.created_at.isoformat(),
            ),
        )
        connection.execute(
            """
            UPDATE screening_sessions
            SET status = ?, completed_at = ?
            WHERE session_id = ?
            """,
            (
                "pending_recapture" if report.overall_risk == "recapture_needed" else "pending_review" i
                report.created_at.isoformat(),
                report.session_id,
            ),
        )

    def get_integrated_report(self, session_id: str) -> IntegratedReportResponse:
        report = self.get_integrated_report_or_none(session_id)
        if report is None:
            raise NotFoundError(f"Integrated report not found: {session_id}")
        return report

    def get_integrated_report_or_none(self, session_id: str) -> IntegratedReportResponse | None:
        with self.db.connect() as connection:
            row = connection.execute(
                """
                SELECT report_id, session_id, title, overall_risk, consistency_level,
                       main_patterns, cross_protocol_evidence, next_action, summary,
                       recommendations, disclaimer, created_at
                FROM integrated_reports
                WHERE session_id = ?
                """,
                (session_id,),
            ).fetchone()
        if row is None:
            return None
        return IntegratedReportResponse(
            report_id=row[0],
            session_id=row[1],
            title=row[2],
            overall_risk=row[3],
            consistency_level=row[4],
```

```
            main_patterns=json.loads(row[5]),
            cross_protocol_evidence=json.loads(row[6]),
            next_action=row[7],
            summary=row[8],
            recommendations=json.loads(row[9]),
            disclaimer=row[10],
            created_at=datetime.fromisoformat(row[11]),
        )

    def _ensure_subject_exists(self, subject_id: str) -> None:
        with self.db.connect() as connection:
            row = connection.execute(
                "SELECT subject_id FROM subjects WHERE subject_id = ?",
                (subject_id,),
            ).fetchone()
        if row is None:
            raise NotFoundError(f"Subject not found: {subject_id}")

    def _get_session_row(self, session_id: str):
        with self.db.connect() as connection:
            row = connection.execute(
                """
                SELECT session_id, subject_id, status, protocols, created_at, completed_at
                FROM screening_sessions
                WHERE session_id = ?
                """,
                (session_id,),
            ).fetchone()
        if row is None:
            raise NotFoundError(f"Screening session not found: {session_id}")
        return row

    def _refresh_session_status(self, session_id: str) -> None:
        results = self.list_protocol_results(session_id)
        if any(result.needs_recapture for result in results):
            status = "pending_recapture"
        elif any(result.needs_review for result in results):
            status = "pending_review"
        else:
            status = "pending_report"
        with self.db.connect() as connection:
            connection.execute(
                "UPDATE screening_sessions SET status = ? WHERE session_id = ?",
                (status, session_id),
            )

    def _protocol_result_from_row(self, row) -> ProtocolResultResponse:
        return ProtocolResultResponse(
            result_id=row[0],
            session_id=row[1],
            protocol=row[2],
            status=row[3],
            capture_quality=row[4],
            metrics=json.loads(row[5]),
            findings=json.loads(row[6]),
            risk_flags=json.loads(row[7]),
            recommendations=json.loads(row[8]),
            needs_recapture=bool(row[9]),
            needs_review=bool(row[10]),
            created_at=datetime.fromisoformat(row[11]),
            updated_at=datetime.fromisoformat(row[12]),
        )
```

```
// features/screening/schemas.py
from datetime import datetime
from typing import Any, Literal
from uuid import uuid4

from pydantic import BaseModel, Field

ProtocolType = Literal["static_posture", "adams_forward_bend", "squat"]
ScreeningStatus = Literal[
    "in_progress",
    "pending_report",
    "completed",
    "pending_recapture",
    "pending_review",
    "archived",
]
ProtocolStatus = Literal[
    "not_started",
    "capturing",
    "captured",
    "analyzed",
    "needs_recapture",
    "needs_review",
]
CaptureQuality = Literal["poor", "acceptable", "good"]
OverallRisk = Literal["low", "attention", "review_required", "recapture_needed"]
NextAction = Literal["pass", "retest_later", "recapture", "manual_review", "professional_evaluation"]
Direction = Literal["left", "right", "forward", "unclear"]
Confidence = Literal["low", "medium", "high"]


class SubjectCreateRequest(BaseModel):
    display_name: str = Field(min_length=1, max_length=80)
    sex: Literal["female", "male", "unknown"] = "unknown"
    age: int | None = Field(default=None, ge=3, le=120)
    height_cm: float | None = Field(default=None, ge=60, le=240)
    notes: str = Field(default="", max_length=500)


class SubjectResponse(SubjectCreateRequest):
    subject_id: str
    created_at: datetime


class ScreeningSessionCreateRequest(BaseModel):
    subject_id: str = Field(min_length=1)
    protocols: list[ProtocolType] = Field(default_factory=lambda: ["static_posture", "adams_forward_bend", "


class ProtocolProgress(BaseModel):
    protocol: ProtocolType
    status: ProtocolStatus


class ScreeningSessionCreateResponse(BaseModel):
    session_id: str
    subject_id: str
    status: ScreeningStatus
    protocols: list[ProtocolProgress]
    created_at: datetime


class ProtocolAnalyzeRequest(BaseModel):
    capture_quality: CaptureQuality
    metrics: dict[str, Any] = Field(default_factory=dict)
```

```
class ProtocolResultResponse(BaseModel):
    result_id: str
    session_id: str
    protocol: ProtocolType
    status: ProtocolStatus
    capture_quality: CaptureQuality
    metrics: dict[str, Any]
    findings: list[str]
    risk_flags: list[str]
    recommendations: list[str]
    needs_recapture: bool
    needs_review: bool
    created_at: datetime
    updated_at: datetime


class CrossProtocolEvidence(BaseModel):
    pattern: str
    protocols: list[ProtocolType]
    direction: Direction | None = None
    evidence: list[str]
    confidence: Confidence


class IntegratedReportResponse(BaseModel):
    report_id: str
    session_id: str
    title: str
    overall_risk: OverallRisk
    consistency_level: Literal["none", "single_protocol", "multi_protocol_consistent"]
    main_patterns: list[str]
    cross_protocol_evidence: list[CrossProtocolEvidence]
    next_action: NextAction
    summary: str
    recommendations: list[str]
    disclaimer: str
    created_at: datetime


class ScreeningSessionDetailResponse(BaseModel):
    session_id: str
    subject_id: str
    status: ScreeningStatus
    protocol_results: list[ProtocolResultResponse]
    integrated_report: IntegratedReportResponse | None
    created_at: datetime
    completed_at: datetime | None = None


class ScreeningSessionSummary(BaseModel):
    session_id: str
    subject_id: str
    subject_display_name: str
    status: ScreeningStatus
    overall_risk: OverallRisk | None = None
    next_action: NextAction | None = None
    completed_protocols: list[ProtocolType]
    created_at: datetime
    completed_at: datetime | None = None


def build_subject_id() -> str:
    return f"subj-{uuid4().hex[:10]}"


def build_screening_session_id() -> str:
    return f"screen-{uuid4().hex[:10]}"
```

```
def build_protocol_result_id(protocol: ProtocolType) -> str:
    prefix = protocol.replace("_", "-")
    return f"res-{prefix}-{uuid4().hex[:8]}"


def build_report_id() -> str:
    return f"report-{uuid4().hex[:10]}"
```

```
// api/routes/camera.py
import base64
import binascii
import logging
import time
from functools import lru_cache
from typing import Iterator

import cv2
import numpy as np
from fastapi import APIRouter, Depends, Response, status
from fastapi.exceptions import HTTPException
from fastapi.responses import StreamingResponse

from app.core.config import get_settings
from app.core.errors import ServiceUnavailableError
from app.features.squat.live_analysis import LiveSquatSessionStore, collect_missing_required_points
from app.features.squat.pose_contract import POSE_CONNECTIONS, MIN_KEYPOINT_VISIBILITY
from app.features.squat.schemas import (
    CameraAnalysisSessionResponse,
    CameraFrameRequest,
    CameraFrameAnalysisResponse,
    CameraKeypointsResponse,
    CameraStatusResponse,
    LiveSquatMetricsResponse,
    PoseKeypoint,
)
from app.features.squat.visual_detection import PoseDetector

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api/v1/camera", tags=["camera"])


@lru_cache
def get_pose_detector() -> PoseDetector:
    settings = get_settings()
    return PoseDetector(model_path=settings.pose_model_path)


def get_pose_detector_dependency() -> PoseDetector:
    return get_pose_detector()


@lru_cache
def get_live_analysis_store() -> LiveSquatSessionStore:
    return LiveSquatSessionStore()


def get_live_analysis_store_dependency() -> LiveSquatSessionStore:
    return get_live_analysis_store()


def close_pose_detector_cache() -> None:
    if get_pose_detector.cache_info().currsize == 0:
        return
```

```
    detector = get_pose_detector()
    detector.close()
    get_pose_detector.cache_clear()


def clear_live_analysis_store_cache() -> None:
    if get_live_analysis_store.cache_info().currsize == 0:
        return

    store = get_live_analysis_store()
    store.clear()
    get_live_analysis_store.cache_clear()


def open_camera_capture() -> cv2.VideoCapture:
    camera_index = get_settings().app_camera_device_index
    capture = cv2.VideoCapture(camera_index)
    if not capture.isOpened():
        capture.release()
        raise ServiceUnavailableError(
            f"Camera device {camera_index} is not available."
        )

    capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    return capture


def build_keypoints_response(detector: PoseDetector, frame) -> CameraKeypointsResponse:
    pose_frame = detector.process_frame(frame)
    return build_keypoints_payload(detector=detector, pose_frame=pose_frame)


def build_keypoints_payload(*, detector: PoseDetector, pose_frame) -> CameraKeypointsResponse:
    return CameraKeypointsResponse(
        has_detection=pose_frame.has_detection,
        frame_width=pose_frame.frame_width,
        frame_height=pose_frame.frame_height,
        keypoints=[
            PoseKeypoint(
                name=keypoint.name,
                x=keypoint.x,
                y=keypoint.y,
                z=keypoint.z,
                visibility=keypoint.visibility,
            )
            for keypoint in pose_frame.keypoints
        ],
        detector_backend=detector.backend,
    )


def build_frame_analysis_payload(
    *,
    detector: PoseDetector,
    pose_frame,
    session_id: str,
    view_mode: str,
    live_metrics: LiveSquatMetricsResponse | None,
) -> CameraFrameAnalysisResponse:
    keypoints = [
        PoseKeypoint(
            name=keypoint.name,
            x=keypoint.x,
            y=keypoint.y,
            z=keypoint.z,
            visibility=keypoint.visibility,
```

```
        )
        for keypoint in pose_frame.keypoints
    ]

    if not pose_frame.has_detection:
        readiness_state = "no_detection"
        missing_keypoints = []
    else:
        missing_keypoints = collect_missing_required_points(pose_frame.keypoints)
        if missing_keypoints:
            readiness_state = "insufficient_pose"
        else:
            readiness_state = "ready" if live_metrics and live_metrics.squat_count > 0 else "capturing"

    return CameraFrameAnalysisResponse(
        session_id=session_id,
        has_detection=pose_frame.has_detection,
        frame_width=pose_frame.frame_width,
        frame_height=pose_frame.frame_height,
        keypoints=keypoints,
        detector_backend=detector.backend,
        live_metrics=live_metrics,
        view_mode=view_mode,
        readiness_state=readiness_state,
        missing_keypoints=missing_keypoints,
        min_keypoint_visibility=MIN_KEYPOINT_VISIBILITY,
        pose_connections=list(POSE_CONNECTIONS),
    )


def decode_frame_data_url(frame_data_url: str) -> np.ndarray:
    encoded_payload = frame_data_url.split(",", 1)[-1].strip()
    if not encoded_payload:
        raise HTTPException(status_code=400, detail="Frame payload is empty.")

    try:
        frame_bytes = base64.b64decode(encoded_payload, validate=True)
    except binascii.Error as error:
        raise HTTPException(status_code=400, detail="Frame payload is not valid base64.") from error

    image_buffer = np.frombuffer(frame_bytes, dtype=np.uint8)
    frame = cv2.imdecode(image_buffer, cv2.IMREAD_COLOR)
    if frame is None:
        raise HTTPException(status_code=400, detail="Frame payload is not a valid image.")

    return frame


def generate_frames(detector: PoseDetector) -> Iterator[bytes]:
    capture = open_camera_capture()

    try:
        while True:
            ok, frame = capture.read()
            if not ok:
                logger.warning("Failed to read frame from camera stream")
                break

            pose_frame = detector.process_frame(frame)
            frame_with_overlay = detector.draw_keypoints(frame.copy(), pose_frame)
            encoded, buffer = cv2.imencode(".jpg", frame_with_overlay)
            if not encoded:
                continue

            yield (
                b"--frame\r\n"
                b"Content-Type: image/jpeg\r\n\r\n"
```

```
                + buffer.tobytes()
                + b"\r\n"
            )
            time.sleep(0.03)
    finally:
        capture.release()
        logger.info("Camera released")


@router.get("/status", response_model=CameraStatusResponse)
def camera_status(
    detector: PoseDetector = Depends(get_pose_detector_dependency),
) -> CameraStatusResponse:
    return CameraStatusResponse(
        available=detector.available,
        backend=detector.backend,
        detail=detector.availability_detail,
    )


@router.post(
    "/sessions",
    response_model=CameraAnalysisSessionResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_camera_session(
    store: LiveSquatSessionStore = Depends(get_live_analysis_store_dependency),
) -> CameraAnalysisSessionResponse:
    return CameraAnalysisSessionResponse(session_id=store.create_session())


@router.delete("/sessions/{session_id}", status_code=status.HTTP_204_NO_CONTENT)
def close_camera_session(
    session_id: str,
    store: LiveSquatSessionStore = Depends(get_live_analysis_store_dependency),
) -> Response:
    store.close_session(session_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)


@router.get("/stream")
def video_stream(
    detector: PoseDetector = Depends(get_pose_detector_dependency),
) -> StreamingResponse:
    if not detector.available:
        raise ServiceUnavailableError(detector.availability_detail)

    return StreamingResponse(
        generate_frames(detector),
        media_type="multipart/x-mixed-replace; boundary=frame",
    )


@router.get("/keypoints", response_model=CameraKeypointsResponse)
def get_keypoints(
    detector: PoseDetector = Depends(get_pose_detector_dependency),
) -> CameraKeypointsResponse:
    if not detector.available:
        raise ServiceUnavailableError(detector.availability_detail)

    capture = open_camera_capture()
    try:
        ok, frame = capture.read()
    finally:
        capture.release()

    if not ok:
```

```
        raise ServiceUnavailableError("Failed to read frame from camera device.")

    return build_keypoints_response(detector, frame)


@router.post("/analyze-frame", response_model=CameraFrameAnalysisResponse)
def analyze_frame(
    payload: CameraFrameRequest,
    detector: PoseDetector = Depends(get_pose_detector_dependency),
    store: LiveSquatSessionStore = Depends(get_live_analysis_store_dependency),
) -> CameraFrameAnalysisResponse:
    if not detector.available:
        raise ServiceUnavailableError(detector.availability_detail)

    frame = decode_frame_data_url(payload.frame_data_url)
    pose_frame = detector.process_frame(frame)
    live_metrics = (
        store.update(payload.session_id, payload.view_mode, pose_frame.keypoints)
        if pose_frame.has_detection
        else None
    )

    return build_frame_analysis_payload(
        detector=detector,
        pose_frame=pose_frame,
        session_id=payload.session_id,
        view_mode=payload.view_mode,
        live_metrics=live_metrics,
    )
```

```
// features/home/pages/HomePage.tsx
import { useTranslation } from "react-i18next";
import { Link } from "react-router-dom";
import { Button, EmptyState, Icon, SurfaceCard } from "../../../shared/components/ui";
import { useHomeDashboard } from "../hooks/useHomeDashboard";

function formatSessionDate(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    month: "short",
    day: "numeric",
    weekday: "short",
  }).format(new Date(dateString));
}

function formatSessionTime(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(dateString));
}

export function HomePage() {
  const { t, i18n } = useTranslation();
  const {
    averageScore,
    error,
    isLoading,
    latestReport,
    latestSession,
    reportError,
    totalSessions,
    trendPoints,
  } = useHomeDashboard();

  const latestDate = latestSession ? formatSessionDate(latestSession.created_at, i18n.language) : "";
  const latestTime = latestSession ? formatSessionTime(latestSession.created_at, i18n.language) : "";
  const dashboardErrorCopy = t("home.dataLoadError");
```

```
  const primaryFinding =
    latestReport?.findings[0] ??
    (reportError ? t("home.reportUnavailable") : t("home.primaryFindingEmpty"));
  const nextRecommendation =
    latestReport?.recommendations[0] ??
    (reportError ? t("home.reportUnavailable") : t("home.recommendationEmpty"));

  return (
    <div className="home-page dashboard-page">
      <section className="dashboard-hero">
        <div className="dashboard-hero-copy">
          <span className="dashboard-hero-pill">{t("home.heroPill")}</span>
          <h1 className="dashboard-hero-title">{t("home.title")}</h1>
          <p className="dashboard-hero-description">{t("home.subtitle")}</p>
          <Link to="/assessment" className="dashboard-hero-button">
            {t("home.startAssessment")}
            <Icon name="arrow_forward" className="dashboard-hero-button-icon" filled />
          </Link>
        </div>

        <div className="dashboard-hero-image-wrap" aria-hidden="true">
          <img
            alt="Squat training"
            className="dashboard-hero-image"
            src="https://lh3.googleusercontent.com/aida-public/AB6AXuBh91cz71Scy4Qrh2oxD1qaiAY3OqGDP41u2PMoc
          />
        </div>
      </section>

      <section className="dashboard-grid">
        <div className="dashboard-column dashboard-column-primary">
          <div className="dashboard-section-heading">{t("home.latestSection")}</div>
          {isLoading ? (
            <SurfaceCard variant="lowest" padding="large">
              <p className="section-caption">{t("common.loading")}</p>
            </SurfaceCard>
          ) : error ? (
            <SurfaceCard variant="low" padding="medium" className="error-card">
              <div className="error-content">
                <Icon name="error" />
                <p>{error}</p>
              </div>
            </SurfaceCard>
          ) : latestSession ? (
            <article className="dashboard-card dashboard-card-session">
              <div className="dashboard-session-header">
                <div>
                  <p className="dashboard-session-date">{latestDate}</p>
                  <h2 className="dashboard-session-title">{t("home.latestTitle")}</h2>
                </div>
                <span className="dashboard-score-badge">{t("home.latestBadge")}</span>
              </div>

              <div className="dashboard-score-row">
                <div className="dashboard-score-block">
                  <p className="dashboard-score-label">{t("home.latestScoreLabel")}</p>
                  <div className="dashboard-score-value">
                    {latestSession.overall_score}
                    <span>/100</span>
                  </div>
                </div>
                <div className="dashboard-score-divider" />
                <div className="dashboard-score-block">
                  <p className="dashboard-score-label">{t("home.latestCountLabel")}</p>
                  <div className="dashboard-score-value dashboard-score-value-dark">
                    {latestSession.squat_count}
                  </div>
```

```
            </div>
          </div>

          <div className="dashboard-session-summary-block">
            <p className="dashboard-session-summary-label">{t("home.latestSummaryLabel")}</p>
            <p className="dashboard-session-summary">{latestSession.summary}</p>
          </div>

          <div className="dashboard-metric-stack">
            <div className="dashboard-metric-row">
              <div className="dashboard-metric-left">
                <Icon
                  name="schedule"
                  className="dashboard-metric-icon"
                  color="var(--color-brand-primary-teal-500)"
                />
                <span className="dashboard-metric-name">{t("home.latestTimeLabel")}</span>
              </div>
              <span className="dashboard-metric-value">{latestTime}</span>
            </div>
          </div>
        </article>
      ) : (
        <article className="dashboard-card dashboard-card-session">
          <EmptyState
            icon="query_stats"
            title={t("home.emptyTitle")}
            description={t("home.emptyDesc")}
            action={
              <Button variant="secondary" icon="play_arrow" to="/assessment">
                {t("home.startAssessment")}
              </Button>
            }
          />
        </article>
      )}
    </div>

    <div className="dashboard-column dashboard-column-secondary">
      <div className="dashboard-section-header">
        <div className="dashboard-section-heading">{t("home.recentSection")}</div>
        <Link to="/history" className="dashboard-link-button">
          {t("home.recentViewAll")}
        </Link>
      </div>
      <div className="dashboard-activity-grid">
        <article className="dashboard-card dashboard-activity-card">
          <div className="dashboard-activity-header">
            <div className="dashboard-activity-icon">
              <Icon name="fitness_center" color="var(--color-brand-primary-teal-500)" />
            </div>
            <div>
              <h3 className="dashboard-activity-title">{t("home.recentTrendTitle")}</h3>
              <p className="dashboard-activity-meta">{t("home.recentTrendMeta")}</p>
            </div>
          </div>
          <div className={`dashboard-chart ${trendPoints.length === 0 ? "dashboard-chart-empty" : ""}`}>
            {trendPoints.length > 0 ? (
              <div className="dashboard-chart-bars">
                {trendPoints.map((point, index) => (
                  <span
                    key={point.sessionId}
                    className={`dashboard-chart-bar dashboard-chart-bar-${(index % 5) + 1}`}
                    style={{ height: `${point.height}%` }}
                    title={`${t("history.score")}: ${point.score}`}
                  />
                ))}
```

```
                </div>
              ) : (
                <p className="dashboard-empty-copy">
                  {error ? dashboardErrorCopy : t("home.recentTrendEmpty")}
                </p>
              )}
            </div>
            <div className="dashboard-activity-footer">
              <span>{t("home.totalSessionsLabel")}: {totalSessions}</span>
              <span className="dashboard-activity-accent">
                {t("home.averageScoreLabel")}: {averageScore ?? "--"}
              </span>
            </div>
          </article>

          <article className="dashboard-card dashboard-activity-card">
            <div className="dashboard-activity-header">
              <div className="dashboard-activity-icon">
                <Icon name="history" color="var(--color-brand-primary-teal-500)" />
              </div>
              <div>
                <h3 className="dashboard-activity-title">{t("home.reportSnapshotTitle")}</h3>
                <p className="dashboard-activity-meta">
                  {latestSession ? `${latestDate} | ${latestTime}` : t("home.reportSnapshotMeta")}
                </p>
              </div>
            </div>
            <div className="dashboard-data-list">
              {latestSession ? (
                <>
                  <div className="dashboard-data-list-row">
                    <span className="dashboard-data-list-label">{t("home.latestSummaryLabel")}</span>
                    <span className="dashboard-data-list-value">{latestSession.summary}</span>
                  </div>
                  <div className="dashboard-data-list-row">
                    <span className="dashboard-data-list-label">{t("home.primaryFindingLabel")}</span>
                    <span className="dashboard-data-list-value">{primaryFinding}</span>
                  </div>
                </>
              ) : (
                <p className="dashboard-empty-copy">
                  {error ? dashboardErrorCopy : t("home.reportSnapshotEmpty")}
                </p>
              )}
            </div>
          </article>
        </div>

        <article className="dashboard-insight">
          <div className="dashboard-insight-icon">
            <Icon name="tips_and_updates" size="small" color="var(--color-pure-white)" filled />
          </div>
          <div>
            <h3 className="dashboard-insight-title">{t("home.aiTipTitle")}</h3>
            <p className="dashboard-insight-copy">
              {latestSession ? nextRecommendation : (error ? dashboardErrorCopy : t("home.recommendationEm
            </p>
          </div>
        </article>
      </div>
    </section>
  </div>
 );
}
```

```
// features/home/hooks/useHomeDashboard.ts
import { useEffect, useMemo, useState } from "react";
import { apiClient } from "../../../shared/api/client";
import type { ReportPreviewResponse, SessionSummary } from "../../../shared/types/api";

const MAX_TREND_POINTS = 6;
const MIN_TREND_BAR_HEIGHT = 18;

function getErrorMessage(error: unknown, fallback: string) {
  return error instanceof Error ? error.message : fallback;
}

export interface HomeTrendPoint {
  sessionId: string;
  score: number;
  height: number;
}

function buildTrendPoints(sessions: SessionSummary[]): HomeTrendPoint[] {
  const recentSessions = sessions.slice(0, MAX_TREND_POINTS).reverse();
  const maxScore = recentSessions.reduce(
    (highestScore, session) => Math.max(highestScore, session.overall_score),
    1,
  );

  return recentSessions.map((session) => ({
    sessionId: session.session_id,
    score: session.overall_score,
    height: Math.max(
      MIN_TREND_BAR_HEIGHT,
      Math.round((session.overall_score / maxScore) * 100),
    ),
  }));
}

export function useHomeDashboard() {
  const [sessions, setSessions] = useState<SessionSummary[]>([]);
  const [latestReport, setLatestReport] = useState<ReportPreviewResponse | null>(null);
  const [error, setError] = useState("");
  const [reportError, setReportError] = useState("");
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    let isActive = true;

    async function loadDashboard() {
      setIsLoading(true);

      try {
        const nextSessions = await apiClient.listSessions();
        if (!isActive) {
          return;
        }

        setSessions(nextSessions);
        setError("");

        if (nextSessions.length === 0) {
          setLatestReport(null);
          setReportError("");
          return;
        }

        try {
          const nextReport = await apiClient.previewReport({
            session_id: nextSessions[0].session_id,
          });
```

```
          if (!isActive) {
            return;
          }

          setLatestReport(nextReport);
          setReportError("");
        } catch (caughtError) {
          if (!isActive) {
            return;
          }

          setLatestReport(null);
          setReportError(getErrorMessage(caughtError, "Report unavailable"));
        }
      } catch (caughtError) {
        if (!isActive) {
          return;
        }

        setSessions([]);
        setLatestReport(null);
        setReportError("");
        setError(getErrorMessage(caughtError, "Load failed"));
      } finally {
        if (isActive) {
          setIsLoading(false);
        }
      }
    }

    loadDashboard();

    return () => {
      isActive = false;
    };
  }, []);

  const latestSession = sessions[0] ?? null;
  const averageScore = useMemo(() => {
    if (sessions.length === 0) {
      return null;
    }

    const totalScore = sessions.reduce((sum, session) => sum + session.overall_score, 0);
    return Math.round(totalScore / sessions.length);
  }, [sessions]);

  const trendPoints = useMemo(() => buildTrendPoints(sessions), [sessions]);

  return {
    averageScore,
    error,
    isLoading,
    latestReport,
    latestSession,
    reportError,
    totalSessions: sessions.length,
    trendPoints,
  };
}
// features/history/pages/HistoryPage.tsx
import { useMemo, useState } from "react";
import { useTranslation } from "react-i18next";
import { useNavigate } from "react-router-dom";
import {
  Button,
```

```
  Icon,
  SurfaceCard,
} from "../../../shared/components/ui";
import { HistoryEmptyState } from "../components/HistoryEmptyState";
import { HistoryList } from "../components/HistoryList";
import { HistoryToolbar } from "../components/HistoryToolbar";
import { useSessionSummaries } from "../hooks/useSessionSummaries";
import { exportSessionSummaries, filterSessions } from "../lib/sessionUtils";

export function HistoryPage() {
  const [searchQuery, setSearchQuery] = useState("");
  const { t, i18n } = useTranslation();
  const navigate = useNavigate();
  const { sessions, error, isLoading } = useSessionSummaries();

  const filteredSessions = useMemo(
    () => filterSessions(sessions, searchQuery),
    [searchQuery, sessions]
  );

  return (
    <div className="history-page page-stack">
      <section className="history-header">
        <div className="history-title-row">
          <h1 className="page-title">{t("history.title")}</h1>
          <Button
            variant="tertiary"
            icon="download"
            onClick={() => exportSessionSummaries(filteredSessions)}
            disabled={filteredSessions.length === 0}
          >
            {t("history.export")}
          </Button>
        </div>
        <p className="page-subtitle">{t("history.subtitle")}</p>
      </section>

      <HistoryToolbar
        searchQuery={searchQuery}
        searchPlaceholder={t("history.searchSessions")}
        onSearchChange={setSearchQuery}
        onClearSearch={() => setSearchQuery("")}
        clearLabel={t("history.clearSearch")}
      />

      {isLoading ? (
        <SurfaceCard variant="lowest" padding="medium">
          <p className="section-caption">{t("common.loading")}</p>
        </SurfaceCard>
      ) : error ? (
        <SurfaceCard variant="low" padding="medium" className="error-card">
          <div className="error-content">
            <Icon name="error" />
            <p>{error}</p>
          </div>
        </SurfaceCard>
      ) : filteredSessions.length === 0 ? (
        <HistoryEmptyState
          hasSearchQuery={Boolean(searchQuery)}
          title={searchQuery ? t("history.noResults") : t("history.noSessions")}
          description={searchQuery ? t("history.noResultsDesc") : t("history.noSessionsDesc")}
          actionLabel={t("home.startAssessment")}
          clearLabel={t("history.clearSearch")}
          onClearSearch={() => setSearchQuery("")}
        />
      ) : (
        <HistoryList
```

```
          sessions={filteredSessions}
          locale={i18n.language}
          typeLabel={t("history.type")}
          countLabel={t("assessment.squatCount")}
          scoreLabel={t("history.score")}
          onSelectSession={(sessionId) => navigate(`/history/${encodeURIComponent(sessionId)}`)}
        />
      )}
    </div>
  );
}
```

```
// features/history/pages/HistoryDetailPage.tsx
import { useTranslation } from "react-i18next";
import { useParams } from "react-router-dom";
import {
  Button,
  DataBadge,
  Icon,
  SurfaceCard,
} from "../../../shared/components/ui";
import { useSessionReportPreview } from "../hooks/useSessionReportPreview";
import { exportSessionReport } from "../lib/sessionUtils";

export function HistoryDetailPage() {
  const { sessionId } = useParams<{ sessionId: string }>();
  const decodedSessionId = sessionId ? decodeURIComponent(sessionId) : null;
  const { t } = useTranslation();
  const { report, error, isLoading } = useSessionReportPreview(decodedSessionId);

  return (
    <div className="history-page page-stack">
      <section className="history-header">
        <div className="history-title-row">
          <div className="stack">
            <h1 className="page-title">{report?.title ?? t("history.detailTitle")}</h1>
            <p className="page-subtitle">{t("history.detailSubtitle")}</p>
          </div>
          <div className="history-actions">
            <Button variant="ghost" icon="arrow_back" to="/history">
              {t("history.backToHistory")}
            </Button>
            <Button
              variant="tertiary"
              icon="download"
              onClick={() => report && exportSessionReport(report)}
              disabled={!report}
            >
              {t("history.exportSession")}
            </Button>
          </div>
        </div>
      </section>

      {isLoading ? (
        <SurfaceCard variant="lowest" padding="medium">
          <p className="section-caption">{t("common.loading")}</p>
        </SurfaceCard>
      ) : error || !report ? (
        <SurfaceCard variant="low" padding="medium" className="error-card">
          <div className="error-content">
            <Icon name="error" />
            <p>{error || t("history.reportUnavailable")}</p>
          </div>
        </SurfaceCard>
      ) : (
        <>
```

```
          <section className="assessment-lower-grid">
            <SurfaceCard variant="lowest" padding="large">
              <div className="assessment-surface-head">
                <div>
                  <h2 className="assessment-surface-title">{t("history.summary")}</h2>
                  <p className="assessment-surface-copy">{report.summary}</p>
                </div>
                <DataBadge
                  variant="primary"
                  label={t("history.sessionId")}
                  value={report.session_id}
                />
              </div>
            </SurfaceCard>

            <SurfaceCard variant="high" padding="large">
              <div className="assessment-surface-head">
                <div>
                  <h2 className="assessment-surface-title">{t("history.recommendations")}</h2>
                  <p className="assessment-surface-copy">{t("history.recommendationsDesc")}</p>
                </div>
              </div>
              <ul className="assessment-suggestion-list">
                {report.recommendations.map((recommendation) => (
                  <li key={recommendation}>{recommendation}</li>
                ))}
              </ul>
            </SurfaceCard>
          </section>

          <section className="assessment-grid">
            <SurfaceCard variant="lowest" padding="large">
              <div className="assessment-surface-head">
                <div>
                  <h2 className="assessment-surface-title">{t("history.findings")}</h2>
                  <p className="assessment-surface-copy">{t("history.findingsDesc")}</p>
                </div>
              </div>
              <ul className="assessment-suggestion-list">
                {report.findings.map((finding) => (
                  <li key={finding}>{finding}</li>
                ))}
              </ul>
            </SurfaceCard>

            <SurfaceCard variant="lowest" padding="large">
              <div className="assessment-surface-head">
                <div>
                  <h2 className="assessment-surface-title">{t("history.nextStepTitle")}</h2>
                  <p className="assessment-surface-copy">{t("history.nextStepDesc")}</p>
                </div>
              </div>
              <div className="assessment-cta-row">
                <Button variant="primary" icon="play_arrow" to="/assessment">
                  {t("home.startAssessment")}
                </Button>
                <Button variant="tertiary" icon="history" to="/history">
                  {t("history.backToHistory")}
                </Button>
              </div>
            </SurfaceCard>
          </section>
        </>
      )}
    </div>
  );
}
```

```tsx
// features/history/components/HistoryList.tsx
import type { SessionSummary } from "../../../shared/types/api";
import { RecordItem } from "../../../shared/components/ui";
import { formatSessionDate, formatSessionTime } from "../lib/sessionUtils";

interface HistoryListProps {
  sessions: SessionSummary[];
  locale: string;
  typeLabel: string;
  countLabel: string;
  scoreLabel: string;
  onSelectSession: (sessionId: string) => void;
}

export function HistoryList({
  sessions,
  locale,
  typeLabel,
  countLabel,
  scoreLabel,
  onSelectSession,
}: HistoryListProps) {
  return (
    <section className="history-list">
      {sessions.map((session) => (
        <RecordItem
          key={session.session_id}
          title={session.session_id}
          date={formatSessionDate(session.created_at, locale)}
          time={formatSessionTime(session.created_at, locale)}
          type={typeLabel}
          count={session.squat_count}
          score={session.overall_score}
          countLabel={countLabel}
          scoreLabel={scoreLabel}
          onClick={() => onSelectSession(session.session_id)}
        />
      ))}
    </section>
  );
}
```

```tsx
// features/history/components/HistoryToolbar.tsx
import { Button, SearchInput } from "../../../shared/components/ui";

interface HistoryToolbarProps {
  searchQuery: string;
  searchPlaceholder: string;
  onSearchChange: (value: string) => void;
  onClearSearch: () => void;
  clearLabel: string;
}

export function HistoryToolbar({
  searchQuery,
  searchPlaceholder,
  onSearchChange,
  onClearSearch,
  clearLabel,
}: HistoryToolbarProps) {
  return (
    <section className="history-filters">
      <SearchInput
        placeholder={searchPlaceholder}
        value={searchQuery}
        onChange={onSearchChange}
        className="search-input-flex"
        ariaLabel={searchPlaceholder}
```

```
      />
      {searchQuery ? (
        <Button variant="ghost" icon="close" onClick={onClearSearch}>
          {clearLabel}
        </Button>
      ) : null}
    </section>
  );
}
```

```
// features/history/components/HistoryEmptyState.tsx
import { Button, EmptyState } from "../../../shared/components/ui";

interface HistoryEmptyStateProps {
  hasSearchQuery: boolean;
  title: string;
  description: string;
  actionLabel: string;
  clearLabel: string;
  onClearSearch: () => void;
}

export function HistoryEmptyState({
  hasSearchQuery,
  title,
  description,
  actionLabel,
  clearLabel,
  onClearSearch,
}: HistoryEmptyStateProps) {
  return (
    <EmptyState
      icon="history"
      title={title}
      description={description}
      action={
        hasSearchQuery ? (
          <Button variant="ghost" icon="close" onClick={onClearSearch}>
            {clearLabel}
          </Button>
        ) : (
          <Button variant="primary" icon="play_arrow" to="/assessment">
            {actionLabel}
          </Button>
        )
      }
    />
  );
}
```

```
// features/history/hooks/useSessionSummaries.ts
import { useEffect, useState } from "react";
import { apiClient } from "../../../shared/api/client";
import type { SessionSummary } from "../../../shared/types/api";

function getErrorMessage(error: unknown, fallback: string) {
  return error instanceof Error ? error.message : fallback;
}

export function useSessionSummaries() {
  const [sessions, setSessions] = useState<SessionSummary[]>([]);
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    let isActive = true;

    async function loadSessions() {
```

```
      setIsLoading(true);

      try {
        const nextSessions = await apiClient.listSessions();
        if (!isActive) {
          return;
        }

        setSessions(nextSessions);
        setError("");
      } catch (caughtError) {
        if (!isActive) {
          return;
        }

        setError(getErrorMessage(caughtError, "Load failed"));
      } finally {
        if (isActive) {
          setIsLoading(false);
        }
      }
    }

    loadSessions();

    return () => {
      isActive = false;
    };
  }, []);

  return {
    sessions,
    error,
    isLoading,
  };
}
// features/history/hooks/useSessionReportPreview.ts
import { useEffect, useState } from "react";
import { apiClient } from "../../../shared/api/client";
import type { ReportPreviewResponse } from "../../../shared/types/api";

function getErrorMessage(error: unknown, fallback: string) {
  return error instanceof Error ? error.message : fallback;
}

export function useSessionReportPreview(sessionId: string | null) {
  const [report, setReport] = useState<ReportPreviewResponse | null>(null);
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(Boolean(sessionId));

  useEffect(() => {
    if (!sessionId) {
      setReport(null);
      setError("");
      setIsLoading(false);
      return;
    }

    const resolvedSessionId = sessionId;
    let isActive = true;

    async function loadReportPreview() {
      setIsLoading(true);

      try {
        const nextReport = await apiClient.previewReport({ session_id: resolvedSessionId });
```

```
        if (!isActive) {
          return;
        }

        setReport(nextReport);
        setError("");
      } catch (caughtError) {
        if (!isActive) {
          return;
        }

        setError(getErrorMessage(caughtError, "Load failed"));
      } finally {
        if (isActive) {
          setIsLoading(false);
        }
      }
    }

    loadReportPreview();

    return () => {
      isActive = false;
    };
  }, [sessionId]);

  return {
    report,
    error,
    isLoading,
  };
}
// features/history/lib/sessionUtils.ts
import type { ReportPreviewResponse, SessionSummary } from "../../../shared/types/api";

export function filterSessions(sessions: SessionSummary[], query: string) {
  const normalizedQuery = query.trim().toLocaleLowerCase();

  if (!normalizedQuery) {
    return sessions;
  }

  return sessions.filter((session) => {
    const values = [session.session_id, session.summary];
    return values.some((value) => value.toLocaleLowerCase().includes(normalizedQuery));
  });
}

export function formatSessionDate(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    year: "numeric",
    month: "short",
    day: "numeric",
  }).format(new Date(dateString));
}

export function formatSessionTime(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(dateString));
}

function downloadJson(filename: string, payload: unknown) {
  const blob = new Blob([JSON.stringify(payload, null, 2)], {
    type: "application/json;charset=utf-8",
```

```
  });
  const url = URL.createObjectURL(blob);
  const anchor = document.createElement("a");

  anchor.href = url;
  anchor.download = filename;
  document.body.appendChild(anchor);
  anchor.click();
  anchor.remove();

  URL.revokeObjectURL(url);
}

export function exportSessionSummaries(sessions: SessionSummary[]) {
  const timestamp = new Date().toISOString().replace(/[:.]/g, "-");
  downloadJson(`squatlab-history-${timestamp}.json`, sessions);
}

export function exportSessionReport(report: ReportPreviewResponse) {
  downloadJson(`squatlab-session-${report.session_id}.json`, report);
}
// features/settings/pages/SettingsPage.tsx
import { useTranslation } from "react-i18next";
import { exportSessionSummaries } from "../../history/lib/sessionUtils";
import { env } from "../../../shared/config/env";
import { Button, EmptyState, Icon, StatCard, SurfaceCard } from "../../../shared/components/ui";
import { useSettingsOverview } from "../hooks/useSettingsOverview";

function formatSessionDateTime(dateString: string, locale: string) {
  return new Intl.DateTimeFormat(locale, {
    year: "numeric",
    month: "short",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(dateString));
}

export function SettingsPage() {
  const { t, i18n } = useTranslation();
  const {
    averageScore,
    cameraStatus,
    dataError,
    health,
    isLoading,
    latestSession,
    serviceError,
    sessions,
    totalSessions,
  } = useSettingsOverview();

  const latestCapturedAt = latestSession
    ? formatSessionDateTime(latestSession.created_at, i18n.language)
    : "";
  const isServiceOnline = health?.status === "ok";
  const isCameraReady = cameraStatus?.available ?? false;

  return (
    <div className="settings-page page-stack">
      <section className="settings-header">
        <h1 className="page-title">{t("settings.title")}</h1>
        <p className="page-subtitle">{t("settings.subtitle")}</p>
      </section>

      {serviceError ? (
```

```
        <SurfaceCard variant="low" padding="medium" className="error-card">
          <div className="error-content">
            <Icon name="error" />
            <p>{serviceError}</p>
          </div>
        </SurfaceCard>
      ) : null}

      <section className="settings-section">
        <h2 className="section-title">{t("settings.environment")}</h2>
        <SurfaceCard variant="lowest" padding="medium" className="env-card">
          <div className="env-item">
            <span className="env-label">{t("settings.apiBaseUrl")}</span>
            <span className="env-value">{env.apiBaseUrl}</span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.serviceName")}</span>
            <span className="env-value">{health?.service ?? "SquatLab API"}</span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.serviceVersion")}</span>
            <span className="env-value">{health?.version ?? "--"}</span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.connectionStatus")}</span>
            <span className={`env-value${isServiceOnline ? "" : " env-value-disabled"}`}>
              {isServiceOnline ? t("settings.statusOnline") : t("settings.statusOffline")}
            </span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.cameraStatus")}</span>
            <span className={`env-value${isCameraReady ? "" : " env-value-disabled"}`}>
              {isCameraReady ? t("settings.cameraReady") : t("settings.cameraUnavailable")}
            </span>
          </div>
          <div className="env-item">
            <span className="env-label">{t("settings.detectorBackend")}</span>
            <span className="env-value">{cameraStatus?.backend ?? "--"}</span>
          </div>
        </SurfaceCard>
      </section>

      <section className="settings-section">
        <h2 className="section-title">{t("settings.savedData")}</h2>
        {isLoading ? (
          <SurfaceCard variant="lowest" padding="medium">
            <p className="section-caption">{t("common.loading")}</p>
          </SurfaceCard>
        ) : dataError ? (
          <SurfaceCard variant="low" padding="medium" className="error-card">
            <div className="error-content">
              <Icon name="error" />
              <p>{dataError}</p>
            </div>
          </SurfaceCard>
        ) : totalSessions === 0 ? (
          <SurfaceCard variant="lowest" padding="large">
            <EmptyState
              icon="query_stats"
              title={t("settings.noRecords")}
              description={t("settings.noRecordsDesc")}
              action={
                <Button variant="secondary" icon="play_arrow" to="/assessment">
                  {t("home.startAssessment")}
                </Button>
              }
            />
```

```
        </SurfaceCard>
      ) : (
        <>
          <div className="settings-grid">
            <StatCard
              label={t("settings.totalSessions")}
              value={totalSessions}
              icon="history"
            />
            <StatCard
              label={t("settings.latestScore")}
              value={latestSession?.overall_score ?? "--"}
              unit={latestSession ? "/100" : undefined}
              icon="query_stats"
              variant="highlight"
            />
            <StatCard
              label={t("settings.averageScore")}
              value={averageScore ?? "--"}
              unit={averageScore !== null ? "/100" : undefined}
              icon="analytics"
              variant="accent"
            />
          </div>

          {latestSession ? (
            <SurfaceCard variant="lowest" padding="medium" className="about-info">
              <h3 className="about-name">{t("settings.latestSummary")}</h3>
              <div className="env-item">
                <span className="env-label">{t("settings.latestCapturedAt")}</span>
                <span className="env-value">{latestCapturedAt}</span>
              </div>
              <p className="about-description">{latestSession.summary}</p>
            </SurfaceCard>
          ) : null}
        </>
      )}
    </section>

    <section className="settings-section">
      <h2 className="section-title">{t("settings.dataActions")}</h2>
      <SurfaceCard variant="low" padding="medium">
        <div className="data-actions">
          <Button
            variant="tertiary"
            icon="download"
            onClick={() => exportSessionSummaries(sessions)}
            disabled={sessions.length === 0}
          >
            {t("settings.exportRecords")}
          </Button>
          <Button variant="tertiary" icon="history" to="/history">
            {t("settings.viewRecords")}
          </Button>
          <Button variant="secondary" icon="play_arrow" to="/assessment">
            {t("home.startAssessment")}
          </Button>
        </div>
      </SurfaceCard>
    </section>

    <section className="settings-section">
      <h2 className="section-title">{t("settings.about")}</h2>
      <SurfaceCard variant="lowest" padding="medium">
        <div className="about-info">
          <p className="about-name">SquatLab</p>
          <p className="about-version">{t("settings.version")}</p>
```

```
            <p className="about-description">{t("settings.description")}</p>
          </div>
        </SurfaceCard>
      </section>
    </div>
  );
}
```

```
// features/settings/hooks/useSettingsOverview.ts
import { useEffect, useMemo, useState } from "react";
import { apiClient } from "../../../shared/api/client";
import type {
  CameraStatusResponse,
  HealthResponse,
  SessionSummary,
} from "../../../shared/types/api";

function getErrorMessage(error: unknown, fallback: string) {
  return error instanceof Error ? error.message : fallback;
}

export function useSettingsOverview() {
  const [health, setHealth] = useState<HealthResponse | null>(null);
  const [cameraStatus, setCameraStatus] = useState<CameraStatusResponse | null>(null);
  const [sessions, setSessions] = useState<SessionSummary[]>([]);
  const [serviceError, setServiceError] = useState("");
  const [dataError, setDataError] = useState("");
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    let isActive = true;

    async function loadOverview() {
      setIsLoading(true);

      const [healthResult, cameraStatusResult, sessionsResult] = await Promise.allSettled([
        apiClient.health(),
        apiClient.cameraStatus(),
        apiClient.listSessions(),
      ]);

      if (!isActive) {
        return;
      }

      const serviceErrors: string[] = [];

      if (healthResult.status === "fulfilled") {
        setHealth(healthResult.value);
      } else {
        setHealth(null);
        serviceErrors.push(getErrorMessage(healthResult.reason, "Service unavailable"));
      }

      if (cameraStatusResult.status === "fulfilled") {
        setCameraStatus(cameraStatusResult.value);
      } else {
        setCameraStatus(null);
        serviceErrors.push(getErrorMessage(cameraStatusResult.reason, "Camera status unavailable"));
      }

      if (sessionsResult.status === "fulfilled") {
        setSessions(sessionsResult.value);
        setDataError("");
      } else {
        setSessions([]);
        setDataError(getErrorMessage(sessionsResult.reason, "Records unavailable"));
```

```
      }

      setServiceError(serviceErrors.join(" "));
      setIsLoading(false);
    }

    loadOverview();

    return () => {
      isActive = false;
    };
  }, []);

  const averageScore = useMemo(() => {
    if (sessions.length === 0) {
      return null;
    }

    const totalScore = sessions.reduce((sum, session) => sum + session.overall_score, 0);
    return Math.round(totalScore / sessions.length);
  }, [sessions]);

  return {
    averageScore,
    cameraStatus,
    dataError,
    health,
    isLoading,
    latestSession: sessions[0] ?? null,
    serviceError,
    sessions,
    totalSessions: sessions.length,
  };
}
```

```
// shared/layout/AppShell.tsx
import { useTranslation } from "react-i18next";
import { Route, Routes } from "react-router-dom";
import { HomePage } from "../../features/home/pages/HomePage";
import { HistoryDetailPage } from "../../features/history/pages/HistoryDetailPage";
import { HistoryPage } from "../../features/history/pages/HistoryPage";
import { SettingsPage } from "../../features/settings/pages/SettingsPage";
import { SquatSessionPage } from "../../features/squat/pages/SquatSessionPage";
import { BottomNavBar, TopAppBar, getDefaultNavItems } from "../components/ui";

export function AppShell() {
  const { t } = useTranslation();

  return (
    <div className="app-shell">
      <TopAppBar title="SquatLab" />

      <main className="app-main">
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/assessment" element={<SquatSessionPage />} />
          <Route path="/history" element={<HistoryPage />} />
          <Route path="/history/:sessionId" element={<HistoryDetailPage />} />
          <Route path="/settings" element={<SettingsPage />} />
        </Routes>
      </main>

      <BottomNavBar
        items={getDefaultNavItems({
          home: t("common.home"),
          assessment: t("common.assessment"),
          history: t("common.history"),
```

```
          settings: t("common.settings"),
        })}
      />
    </div>
  );
}
```

```
// shared/components/Icon.tsx
interface IconProps {
  name: string;
  size?: "small" | "medium" | "large";
  className?: string;
  color?: string;
  filled?: boolean;
}

export function Icon({
  name,
  size = "medium",
  className = "",
  color = "currentColor",
  filled = false,
}: IconProps) {
  const sizeMap = {
    small: "16px",
    medium: "24px",
    large: "32px",
  };

  return (
    <span
      className={`icon material-symbols-outlined ${className}`.trim()}
      style={{
        fontSize: sizeMap[size],
        color,
        fontVariationSettings: filled
          ? "'FILL' 1, 'wght' 500, 'GRAD' 0, 'opsz' 24"
          : "'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 24",
      }}
      aria-hidden="true"
    >
      {name}
    </span>
  );
}
```

```
// shared/components/LanguageSwitcher.tsx
import { useTranslation } from "react-i18next";
import { Button } from "./ui";

interface LanguageSwitcherProps {
  className?: string;
}

export function LanguageSwitcher({ className = "" }: LanguageSwitcherProps) {
  const { i18n } = useTranslation();

  const changeLanguage = (lng: string) => {
    i18n.changeLanguage(lng);
  };

  const currentLanguage = i18n.language;

  return (
    <div className={`language-switcher ${className}`}>
      <Button
        variant={currentLanguage === "zh-CN" ? "primary" : "tertiary"}
        size="small"
```

```
        onClick={() => changeLanguage("zh-CN")}
        className="language-button"
      >
        中文
      </Button>
      <Button
        variant={currentLanguage === "en-US" ? "primary" : "tertiary"}
        size="small"
        onClick={() => changeLanguage("en-US")}
        className="language-button"
      >
        English
      </Button>
    </div>
  );
}
```

```
// shared/components/ui/index.ts
export { TopAppBar, IconButton, Badge } from "./TopAppBar";
export { BottomNavBar, getDefaultNavItems } from "./BottomNavBar";
export { SurfaceCard, GlassCard } from "./SurfaceCard";
export { Button } from "./Button";
export { StatCard, MiniStat } from "./StatCard";
export { ProgressBar, SplitProgressBar } from "./ProgressBar";
export { RecordItem } from "./RecordItem";
export { SearchInput } from "./SearchInput";
export { ActivityCard } from "./ActivityCard";
export { EmptyState } from "./EmptyState";
export { InsightCard } from "./InsightCard";
export { DataBadge } from "./DataBadge";
export { LanguageSwitcher } from "../LanguageSwitcher";
export { Icon } from "../Icon";
```

```
// shared/components/ui/Button.tsx
import type { MouseEvent, ReactNode } from "react";
import { Link } from "react-router-dom";
import { Icon } from "../Icon";

interface ButtonProps {
  children: ReactNode;
  variant?: "primary" | "secondary" | "tertiary" | "ghost";
  size?: "small" | "medium" | "large";
  icon?: string;
  iconPosition?: "left" | "right";
  disabled?: boolean;
  loading?: boolean;
  fullWidth?: boolean;
  onClick?: () => void;
  type?: "button" | "submit" | "reset";
  className?: string;
  to?: string;
  href?: string;
  target?: string;
  rel?: string;
  ariaLabel?: string;
}

export function Button({
  children,
  variant = "primary",
  size = "medium",
  icon,
  iconPosition = "left",
  disabled = false,
  loading = false,
  fullWidth = false,
  onClick,
  type = "button",
```

```
  className = "",
  to,
  href,
  target,
  rel,
  ariaLabel,
}: ButtonProps) {
  const buttonClass = [
    "btn",
    `btn-${variant}`,
    `btn-${size}`,
    fullWidth ? "btn-full-width" : "",
    loading ? "btn-loading" : "",
    className,
  ]
    .filter(Boolean)
    .join(" ");

  const iconElement = icon && (
    <Icon name={icon} className="btn-icon" />
  );

  const isUnavailable = disabled || loading;
  const buttonContent = loading ? (
    <Icon name="sync" className="btn-icon spin" />
  ) : (
    <>
      {icon && iconPosition === "left" && iconElement}
      {children}
      {icon && iconPosition === "right" && iconElement}
    </>
  );

  const handleUnavailableClick = (event: MouseEvent<HTMLElement>) => {
    if (!isUnavailable) {
      onClick?.();
      return;
    }

    event.preventDefault();
  };

  if (to) {
    return (
      <Link
        to={to}
        className={buttonClass}
        aria-disabled={isUnavailable}
        aria-label={ariaLabel}
        onClick={handleUnavailableClick}
      >
        {buttonContent}
      </Link>
    );
  }

  if (href) {
    return (
      <a
        href={href}
        className={buttonClass}
        target={target}
        rel={rel}
        aria-disabled={isUnavailable}
        aria-label={ariaLabel}
        onClick={handleUnavailableClick}
      >
```

```
        {buttonContent}
      </a>
    );
  }

  return (
    <button
      type={type}
      className={buttonClass}
      disabled={isUnavailable}
      onClick={onClick}
      aria-label={ariaLabel}
    >
      {buttonContent}
    </button>
  );
}
```

```
// shared/components/ui/TopAppBar.tsx
import React from "react";
import { Link } from "react-router-dom";
import { Icon } from "../Icon";

interface TopAppBarProps {
  title: string;
}

export function TopAppBar({ title }: TopAppBarProps) {
  return (
    <header className="top-app-bar">
      <div className="top-app-bar-content">
        <Link to="/" className="top-app-bar-brand">
          <Icon
            name="analytics"
            className="top-app-bar-brand-icon"
            color="var(--color-brand-primary-teal-500)"
          />
          <h1 className="top-app-bar-title">{title}</h1>
        </Link>
        <Link to="/settings" className="top-app-bar-settings" aria-label="Settings">
          <Icon
            name="settings"
            className="top-app-bar-settings-icon"
            color="var(--color-neutral-paper-800)"
          />
        </Link>
      </div>
      <div className="top-app-bar-divider" />
    </header>
  );
}

interface IconButtonProps {
  icon: string;
  onClick?: () => void;
  ariaLabel?: string;
  active?: boolean;
}

export function IconButton({ icon, onClick, ariaLabel, active }: IconButtonProps) {
  return (
    <button
      type="button"
      className={`icon-button ${active ? "active" : ""}`}
      onClick={onClick}
      aria-label={ariaLabel}
    >
```

```
      <Icon name={icon} />
    </button>
  );
}

interface BadgeProps {
  children: React.ReactNode;
  variant?: "primary" | "secondary" | "success" | "warning" | "error";
}

export function Badge({ children, variant = "primary" }: BadgeProps) {
  return <span className={`badge badge-${variant}`}>{children}</span>;
}
// shared/components/ui/BottomNavBar.tsx
import { NavLink } from "react-router-dom";
import { Icon } from "../Icon";

interface NavItem {
  path: string;
  label: string;
  icon: string;
  activeIcon?: string;
}

interface BottomNavBarProps {
  items: NavItem[];
}

export function BottomNavBar({ items }: BottomNavBarProps) {
  return (
    <nav className="bottom-nav-bar">
      {items.map((item) => (
        <NavLink
          key={item.path}
          to={item.path}
          className={({ isActive }) => `nav-item ${isActive ? "nav-item-active" : ""}`}
          end={item.path === "/"}
        >
          {({ isActive }) => (
            <>
              <Icon
                name={isActive && item.activeIcon ? item.activeIcon : item.icon}
                className="nav-icon"
                filled={isActive}
              />
              <span className="nav-label">{item.label}</span>
            </>
          )}
        </NavLink>
      ))}
    </nav>
  );
}

export function getDefaultNavItems(labels: {
  home: string;
  assessment: string;
  history: string;
  settings: string;
}): NavItem[] {
  return [
    { path: "/", label: labels.home, icon: "home", activeIcon: "home_app_logo" },
    { path: "/assessment", label: labels.assessment, icon: "fitness_center" },
    { path: "/history", label: labels.history, icon: "history" },
    { path: "/settings", label: labels.settings, icon: "person" },
  ];
```

```
}
// shared/components/ui/SurfaceCard.tsx
import { ReactNode } from "react";

interface SurfaceCardProps {
  children: ReactNode;
  variant?: "lowest" | "low" | "high" | "container";
  padding?: "none" | "small" | "medium" | "large";
  className?: string;
  onClick?: () => void;
  hoverable?: boolean;
}

export function SurfaceCard({
  children,
  variant = "lowest",
  padding = "medium",
  className = "",
  onClick,
  hoverable = false,
}: SurfaceCardProps) {
  const cardClass = [
    "surface-card",
    `surface-card-${variant}`,
    `surface-card-padding-${padding}`,
    hoverable ? "surface-card-hoverable" : "",
    onClick ? "surface-card-clickable" : "",
    className,
  ]
    .filter(Boolean)
    .join(" ");

  if (onClick) {
    return (
      <button type="button" className={cardClass} onClick={onClick}>
        {children}
      </button>
    );
  }

  return <div className={cardClass}>{children}</div>;
}

interface GlassCardProps {
  children: ReactNode;
  opacity?: number;
  blur?: number;
  className?: string;
}

export function GlassCard({
  children,
  opacity = 80,
  blur = 20,
  className = "",
}: GlassCardProps) {
  return (
    <div
      className={`glass-card ${className}`}
      style={{
        background: `rgba(255, 255, 255, ${opacity / 100})`,
        backdropFilter: `blur(${blur}px)`,
      }}
    >
      {children}
    </div>
```

```
  );
}

// shared/components/ui/StatCard.tsx
import { Icon } from "../Icon";

interface StatCardProps {
  label: string;
  value: string | number;
  unit?: string;
  trend?: "up" | "down" | "neutral";
  trendValue?: string;
  icon?: string;
  variant?: "default" | "highlight" | "accent";
}

export function StatCard({
  label,
  value,
  unit,
  trend,
  trendValue,
  icon,
  variant = "default",
}: StatCardProps) {
  const trendIcon = trend ? (trend === "up" ? "trending_up" : trend === "down" ? "trending_down" : "trending

  return (
    <div className={`stat-card stat-card-${variant}`}>
      <div className="stat-card-header">
        {icon && (
          <div className="stat-card-icon-wrapper">
            <Icon name={icon} className="stat-card-icon" />
          </div>
        )}
        <span className="stat-card-label">{label}</span>
      </div>
      <div className="stat-card-value-wrapper">
        <span className="stat-card-value">{value}</span>
        {unit && <span className="stat-card-unit">{unit}</span>}
      </div>
      {trend && trendValue && trendIcon && (
        <div className={`stat-card-trend stat-card-trend-${trend}`}>
          <Icon name={trendIcon} className="trend-icon" />
          <span className="trend-value">{trendValue}</span>
        </div>
      )}
    </div>
  );
}

interface MiniStatProps {
  label: string;
  value: string | number;
}

export function MiniStat({ label, value }: MiniStatProps) {
  return (
    <div className="mini-stat">
      <p className="mini-stat-label">{label}</p>
      <p className="mini-stat-value">{value}</p>
    </div>
  );
}
```

```
// shared/components/ui/ProgressBar.tsx
interface ProgressBarProps {
  value: number;
  max?: number;
  label?: string;
  showValue?: boolean;
  variant?: "primary" | "secondary" | "success" | "warning" | "error";
  size?: "small" | "medium" | "large";
  className?: string;
}

export function ProgressBar({
  value,
  max = 100,
  label,
  showValue = false,
  variant = "primary",
  size = "medium",
  className = "",
}: ProgressBarProps) {
  const percentage = Math.min(100, Math.max(0, (value / max) * 100));

  return (
    <div className={`progress-bar-container ${className}`}>
      {(label || showValue) && (
        <div className="progress-bar-header">
          {label && <span className="progress-bar-label">{label}</span>}
          {showValue && (
            <span className="progress-bar-value">{Math.round(percentage)}%</span>
          )}
        </div>
      )}
      <div className={`progress-bar progress-bar-${size}`}>
        <div
          className={`progress-bar-fill progress-bar-${variant}`}
          style={{ width: `${percentage}%` }}
          role="progressbar"
          aria-valuenow={value}
          aria-valuemin={0}
          aria-valuemax={max}
        />
      </div>
    </div>
  );
}

interface SplitProgressBarProps {
  leftValue: number;
  rightValue: number;
  leftLabel?: string;
  rightLabel?: string;
  showLabels?: boolean;
}

export function SplitProgressBar({
  leftValue,
  rightValue,
  leftLabel,
  rightLabel,
  showLabels = false,
}: SplitProgressBarProps) {
  const total = leftValue + rightValue;
  const leftPercentage = total > 0 ? (leftValue / total) * 100 : 50;
  const rightPercentage = total > 0 ? (rightValue / total) * 100 : 50;

  return (
    <div className="split-progress-bar">
```

```
      {showLabels && (
        <div className="split-progress-labels">
          <span>{leftLabel || `${Math.round(leftPercentage)}%`}</span>
          <span>{rightLabel || `${Math.round(rightPercentage)}%`}</span>
        </div>
      )}
      <div className="split-progress-track">
        <div
          className="split-progress-left"
          style={{ width: `${leftPercentage}%` }}
        />
        <div
          className="split-progress-right"
          style={{ width: `${rightPercentage}%` }}
        />
      </div>
    </div>
  );
}
```

```
// shared/components/ui/RecordItem.tsx
import { Icon } from "../Icon";

interface RecordItemProps {
  title: string;
  date: string;
  time?: string;
  type?: string;
  count?: number;
  score?: number;
  countLabel?: string;
  scoreLabel?: string;
  scoreTrend?: "up" | "down" | "neutral";
  onClick?: () => void;
}

export function RecordItem({
  title,
  date,
  time,
  type,
  count,
  score,
  countLabel,
  scoreLabel,
  scoreTrend = "neutral",
  onClick,
}: RecordItemProps) {
  const metadata = [type, time].filter(Boolean).join(" / ");

  return (
    <button type="button" className="record-item" onClick={onClick}>
      <div className="record-item-left">
        <div className="record-item-icon">
          <Icon name="calendar_today" />
        </div>
        <div className="record-item-info">
          <p className="record-item-title">{title}</p>
          <p className="record-item-date">{date}</p>
          {metadata ? <p className="record-item-type">{metadata}</p> : null}
        </div>
      </div>
      <div className="record-item-right">
        {count !== undefined ? (
          <div className="record-item-stat hidden-sm">
            <p className="record-item-stat-label">{countLabel}</p>
            <p className="record-item-stat-value">{count}</p>
```

```
          </div>
        ) : null}
        {score !== undefined ? (
          <div className="record-item-score">
            <p className="record-item-stat-label">{scoreLabel}</p>
            <div className="record-item-score-wrapper">
              <span className="record-item-score-value">{score}</span>
              {scoreTrend !== "neutral" ? (
                <Icon
                  name={scoreTrend === "up" ? "trending_up" : "trending_down"}
                  className={`record-item-trend ${scoreTrend === "up" ? "trend-up" : "trend-down"}`}
                />
              ) : null}
            </div>
          </div>
        ) : null}
        <Icon name="chevron_right" className="record-item-chevron" />
      </div>
    </button>
  );
}
```

```
// shared/components/ui/SearchInput.tsx
import type { ChangeEvent, KeyboardEvent } from "react";
import { Icon } from "../Icon";

interface SearchInputProps {
  placeholder?: string;
  value?: string;
  onChange?: (value: string) => void;
  onSubmit?: (value: string) => void;
  className?: string;
  ariaLabel?: string;
}

export function SearchInput({
  placeholder = "",
  value = "",
  onChange,
  onSubmit,
  className = "",
  ariaLabel,
}: SearchInputProps) {
  const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
    onChange?.(event.target.value);
  };

  const handleKeyDown = (event: KeyboardEvent<HTMLInputElement>) => {
    if (event.key === "Enter" && onSubmit) {
      onSubmit(value);
    }
  };

  return (
    <div className={`search-input-wrapper ${className}`}>
      <Icon name="search" className="search-icon" />
      <input
        type="search"
        className="search-input"
        placeholder={placeholder}
        value={value}
        onChange={handleChange}
        onKeyDown={handleKeyDown}
        aria-label={ariaLabel ?? placeholder}
      />
    </div>
  );
```

```
}
// shared/components/ui/ActivityCard.tsx
import { Icon } from "../Icon";

interface ActivityCardProps {
  title: string;
  date: string;
  duration: string;
  icon: string;
  stats?: {
    label: string;
    value: string;
  }[];
  chartData?: number[];
}

export function ActivityCard({
  title,
  date,
  duration,
  icon,
  stats = [],
  chartData,
}: ActivityCardProps) {
  return (
    <div className="activity-card">
      <div className="activity-card-header">
        <div className="activity-card-icon">
          <Icon name={icon} />
        </div>
        <div className="activity-card-info">
          <h5 className="activity-card-title">{title}</h5>
          <p className="activity-card-meta">
            {date}  {duration}
          </p>
        </div>
      </div>

      {chartData && chartData.length > 0 && (
        <div className="activity-card-chart">
          <div className="chart-bars">
            {chartData.map((value, index) => (
              <div
                key={index}
                className="chart-bar"
                style={{ height: `${value}%` }}
              />
            ))}
          </div>
        </div>
      )}

      {stats.length > 0 && (
        <div className="activity-card-stats">
          {stats.map((stat, index) => (
            <span key={index} className="activity-card-stat">
              {stat.label}: <strong>{stat.value}</strong>
            </span>
          ))}
        </div>
      )}
    </div>
  );
}
```

```tsx
// shared/components/ui/EmptyState.tsx
import type { ReactNode } from "react";
import { Icon } from "../Icon";

interface EmptyStateProps {
  icon?: string;
  title: string;
  description?: string;
  action?: ReactNode;
}

export function EmptyState({
  icon = "inbox",
  title,
  description,
  action,
}: EmptyStateProps) {
  return (
    <div className="empty-state">
      <Icon name={icon} className="empty-state-icon" />
      <h4 className="empty-state-title">{title}</h4>
      {description ? <p className="empty-state-description">{description}</p> : null}
      {action ? <div className="empty-state-action">{action}</div> : null}
    </div>
  );
}
// shared/components/ui/InsightCard.tsx
import { Button } from "./Button";
import { Icon } from "../Icon";

interface InsightCardProps {
  icon?: string;
  iconBg?: "primary" | "secondary" | "tertiary";
  title: string;
  description: string;
  action?: {
    label: string;
    icon?: string;
    onClick?: () => void;
    to?: string;
    href?: string;
  };
  variant?: "default" | "dashed";
}

export function InsightCard({
  icon = "tips_and_updates",
  iconBg = "primary",
  title,
  description,
  action,
  variant = "default",
}: InsightCardProps) {
  return (
    <div className={`insight-card insight-card-${variant}`}>
      <div className="insight-card-content">
        <div className={`insight-card-icon icon-bg-${iconBg}`}>
          <Icon name={icon} />
        </div>
        <div className="insight-card-text">
          <h4 className="insight-card-title">{title}</h4>
          <p className="insight-card-description">{description}</p>
          {action && (action.onClick || action.to || action.href) ? (
            <Button
              variant="ghost"
              size="small"
              className="insight-card-action"
```

```
              onClick={action.onClick}
              to={action.to}
              href={action.href}
              icon={action.icon}
              iconPosition="right"
            >
              {action.label}
            </Button>
          ) : null}
        </div>
      </div>
    </div>
  );
}
```

```
// shared/components/ui/DataBadge.tsx
import { Icon } from "../Icon";

interface DataBadgeProps {
  icon?: string;
  label: string;
  value: string | number;
  variant?: "primary" | "secondary" | "success" | "warning" | "error";
}

export function DataBadge({
  icon,
  label,
  value,
  variant = "primary",
}: DataBadgeProps) {
  return (
    <div className={`data-badge data-badge-${variant}`}>
      {icon ? <Icon name={icon} className="data-badge-icon" /> : null}
      <span className="data-badge-label">{label}</span>
      <span className="data-badge-value">{value}</span>
    </div>
  );
}
```

```
// shared/config/env.ts
const fallbackBaseUrl = "http://localhost:8000";

export const env = {
  apiBaseUrl: import.meta.env.VITE_API_BASE_URL || fallbackBaseUrl,
};
```

```
// shared/i18n/index.ts
import i18n from "i18next";
import { initReactI18next } from "react-i18next";
import zhCN from "./zh-CN.json";
import enUS from "./en-US.json";

const resources = {
  "zh-CN": {
    translation: zhCN,
  },
  "en-US": {
    translation: enUS,
  },
};

i18n.use(initReactI18next).init({
  resources,
  lng: "zh-CN",
  fallbackLng: "zh-CN",
  interpolation: {
    escapeValue: false,
  },
```

```
});

export default i18n;
```